

SEAEVO: Advancing Algorithm Discovery with Strategy Space Evolution

Sichun Luo¹, Yi Huang², Haochen Luo¹, Fengyuan Liu¹, Guanzhi Deng³, Lei Li¹
 Qinghua Yao¹, Zefa Hu², Junlan Feng², Qi Liu¹

¹The University of Hong Kong ²JIUTIAN Research, China Mobile

³City University of Hong Kong

sichunluo2@gmail.com

Abstract

Large Language Model (LLM)-guided evolutionary search is increasingly used for automated algorithm discovery, yet most current methods track search progress primarily through executable programs and scalar fitness. Even when natural-language reasoning is used through heuristic descriptions or reflection, it typically remains transient mutation context or unstructured memory, rather than organized as persistent population-level state over strategic directions. As a result, evolutionary search can struggle to distinguish syntactically different implementations of the same idea, preserve lower-fitness but strategically promising directions, or detect when an entire family of strategies has saturated.

We introduce SEAEVO, a modular strategy-space layer that turns language-level strategic reasoning into first-class population-level evolutionary state in LLM-driven program search. SEAEVO represents each candidate program with an explicit natural-language strategy, clusters the archive by strategy semantics, retrieves behaviorally complementary inspirations, and periodically navigates the strategy landscape to avoid saturated directions. Without modifying the underlying evolutionary algorithms, SEAEVO improves existing evolutionary backbones across algorithm discovery, systems optimization, and agent-scaffold design tasks in most settings. Across four systems benchmarks, SEAEVO achieves a 20.6% average relative improvement, with the best single run on Prism scoring 3× higher. These results suggest that persistent strategy representations provide a practical mechanism for improving the effectiveness and cost-efficiency of LLM-guided evolutionary search, pointing toward compound AI systems whose search capabilities benefit from the structured accumulation and reuse of algorithmic strategies.

1 Introduction

Automatically discovering high-performing algorithms through Large Language Model (LLM)-driven evolutionary search has emerged as a promising paradigm (Romera-Paredes et al., 2024; Novikov et al., 2025). By pairing LLMs as mutation operators with programmatic evaluators, recent LLM-driven evolutionary systems iteratively propose, execute, and refine candidate programs, achieving strong results in domains where search spaces are combinatorially large and hand-crafted heuristics are costly to design (Liu et al., 2024; Ye et al., 2024; Liu et al., 2025, 2026a).

Despite this progress, most LLM-driven evolutionary systems still represent search state primarily as executable programs and scalar fitness values (Romera-Paredes et al., 2024; Novikov et al., 2025; Sharma, 2025). Such archives support evaluation and selection, but they provide only a limited view of search progress: different programs may instantiate the same underlying strategy, similar scores may correspond to qualitatively different directions, and low-scoring candidates may encode promising

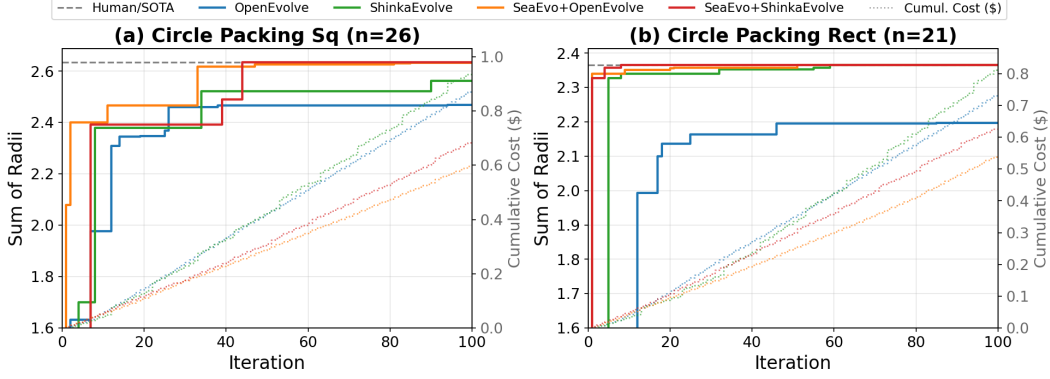


Figure 1: Search trajectories on Circle Packing benchmarks. Solid curves show the best-so-far sum of radii over generations, dotted curves show cumulative API cost, and the gray dashed line denotes the Human/SOTA reference. SEAEVO improves both OpenEvolve and ShinkaEvolve backbones, reaching competitive solutions earlier while incurring lower cumulative cost.

ideas that have not yet been refined. Recent work has introduced richer language-level signals, including natural-language heuristic descriptions (Liu et al., 2024) and reflection over failures (Shinn et al., 2023; Ye et al., 2024; Agrawal et al., 2025). However, these signals are typically used as local prompt context or unstructured memory, rather than as a persistent population-level representation of semantic strategy families.

This strategy-representation gap creates three practical limitations. First, syntactically different programs may implement the same underlying idea, making redundant variants appear as genuine search progress. Second, selection based primarily on scalar fitness may discard lower-scoring candidates that encode strategically useful directions or cover complementary failure modes. Third, per-program fitness provides little visibility into strategy-family dynamics, making it difficult to detect when an entire class of approaches has saturated. These limitations suggest that improving LLM-guided evolution requires not only better mutation operators or evaluators, but also a richer representation of the evolving strategy landscape.

To address this gap, we introduce **SEAEVO (Strategy-space Evolution)**, which elevates natural-language strategy to a first-class population-level representation in LLM-driven evolutionary program search. SEAEVO augments each candidate with an explicit natural-language strategy description and embedding, yielding a dual-space archive organized by both executable programs and semantic strategies. It consists of three coordinated modules: Strategy Articulation (SA), which diagnoses failures and specifies a strategic direction before code generation; Stratified Experience Retrieval (SER), which clusters the archive in strategy space and retrieves behaviorally complementary inspirations; and Strategic Landscape Navigation (SLN), which summarizes effective, saturated, and underexplored strategy families to guide future mutations. These modules turn the archive from a flat record of programs into a navigable map of algorithmic ideas. We evaluate SEAEVO across mathematical algorithm discovery, systems optimization, and agent-scaffold design benchmarks, using diverse backbone LLMs and base evolutionary frameworks. SEAEVO improves the underlying backbones in most settings, with the largest gains on open-ended systems optimization tasks where qualitatively distinct strategy families exist. Beyond this aggregate gain, SEAEVO also improves the search frontier: Figure 1 shows that it reaches competitive solutions in fewer generations while incurring lower cumulative API cost. Case studies in Appendix F further show that strategy-space guidance can redirect evolution away from saturated local refinements toward qualitatively different algorithmic families. These results suggest that persistent strategy representations provide a practical mechanism for improving the effectiveness and efficiency of LLM-guided evolutionary search.

Contributions. Our contributions are as follows: 1) We identify a strategy-representation gap in LLM-guided evolutionary search: existing archives organize search state around programs and scalar fitness scores, but do not maintain a population-level semantic representation of the strategies being explored. 2) We introduce SEAEVO, a modular strategy-space layer that turns natural-language strategies into persistent population-level evolutionary state through strategy descriptions, semantic clustering, behaviorally complementary retrieval, and landscape-level navigation over effective, saturated, and underexplored strategy families. 3) Extensive experiments across mathematical

algorithm discovery, systems optimization, and agent-scaffold design benchmarks demonstrate the effectiveness of SEAEVO. SEAEVO improves multiple open evolutionary backbones in most settings, with a 20.6% average relative improvement across four systems optimization tasks.

2 Related Work

LLM-Driven Evolutionary Algorithm Discovery. Recent work frames algorithm and program improvement as an evolutionary loop in which LLMs act as proposal or mutation operators (Van Stein and Bäck, 2024; Liu et al., 2026b). FunSearch (Romera-Paredes et al., 2024) established this paradigm by pairing an LLM with a programmatic evaluator to evolve mathematical functions, while AlphaEvolve (Novikov et al., 2025) scaled it to full codebases using a Gemini ensemble and a MAP-Elites-inspired program database (Mouret and Clune, 2015). Subsequent systems improve the efficiency and dynamics of this loop through mechanisms such as adaptive parent sampling and novelty rejection in ShinkaEvolve (Lange et al., 2025), island-based genetic operators in CodeEvolve (Assumpção et al., 2025), hierarchical context management and backtracking in PACEvolve (Yan et al., 2026), and adaptive budget and exploration control in AdaEvolve (Cemri et al., 2026). These methods primarily improve how programs are sampled, mutated, selected, or orchestrated. SEAEVO addresses a complementary axis: what the evolutionary state represents. Rather than replacing the base loop, it augments the archive with persistent strategy descriptions, semantic strategy clusters, and LLM-driven landscape navigation, making it compatible with existing evolutionary backbones.

Strategy, Reflection, and Memory in LLM-Based Heuristic Evolution. A closely related line of work introduces language-level reasoning into optimization and evolutionary search (Yuksekgonul et al., 2025; Lee et al., 2025). EoH (Liu et al., 2024) co-evolves natural-language thoughts alongside code; ReEvo (Ye et al., 2024), Reflexion (Shinn et al., 2023), and GEPA (Agrawal et al., 2025) use verbal feedback or reflection to improve future proposals. Other systems maintain reusable insights (Chen et al., 2025), abstractions (Wu et al., 2025), or evolution strategies (Yang et al., 2025b). These works show that natural language can provide useful intermediate structure between scalar feedback and executable code. SEAEVO builds on this direction but differs in how the language signal is organized and used: prior systems typically attach thoughts, reflections, insights, or recommendations to individual candidates, lineages, or textual memories, whereas SEAEVO organizes the full archive into semantic strategy clusters, retrieves inspirations by behavioral complementarity, and tracks strategy-family dynamics at the population level. This makes strategy space a structured object of search rather than only a source of prompt content.

3 Method

3.1 Problem Formulation

We consider the problem of automatically discovering a high-performing program for a given task. Let $f : \mathcal{P} \rightarrow \mathbb{R}$ be an evaluation metric over a space of candidate programs $\mathcal{P}$. The goal is to discover

$$P^* = \arg \max_{P \in \mathcal{P}} f(P), \quad (1)$$

where f measures task-specific fitness. In an evolutionary framework, at generation t an archive $\mathcal{A}_t = \{(P_i, s_i)\}$ with $s_i = f(P_i)$ is maintained under a fixed capacity: an LLM prompt is constructed from selected entries via context builder, a new candidate is generated and evaluated, and then the archive is updated.

Existing frameworks track search progress only through code and scalar fitness, or attach natural-language reflections as transient prompt context without persisting them at the archive level. Neither representation exposes which strategic directions have been explored, which remain underrepresented, and which have already saturated. SEAEVO addresses this by augmenting each archive entry with a persistent strategy description and embedding, organizing the archive as a navigable map of algorithmic ideas rather than a flat record of programs.

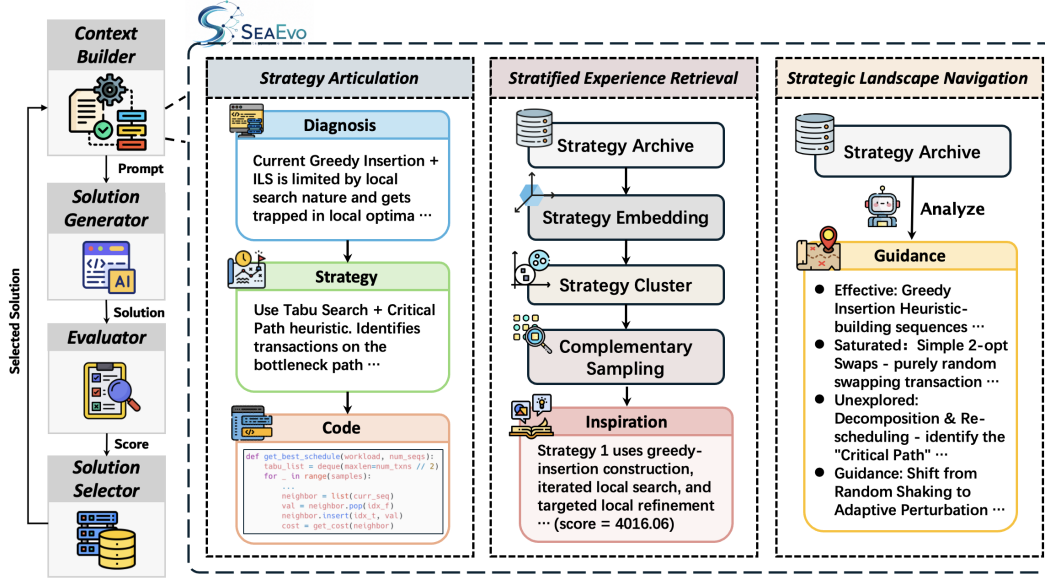


Figure 2: Overview of SEAEVO. SEAEVO replaces the default context builder with a strategy-aware pipeline, while leaving the base loop’s selection and evaluation unchanged. Three modules enrich the evolutionary loop: Strategy Articulation replaces blind mutation with a diagnose-direct-implement pipeline; Stratified Experience Retrieval assembles context from semantically clustered, behaviorally diverse archive entries; Strategic Landscape Navigation periodically diagnoses which strategy families are effective, saturated, or underexplored, and guides future mutations accordingly.

3.2 Framework Overview

Figure 2 illustrates SEAEVO. The method is a modular layer that can be added on top of an existing LLM-driven evolutionary framework: it replaces the framework’s default context builder with a strategy-aware pipeline and augments each archive entry with a persistent strategy description and embedding, while leaving the base loop’s parent selection and evaluation mechanisms unchanged. At each generation, the base framework selects a parent program. SER retrieves strategy-level inspirations from the strategy-augmented archive; SLN provides landscape-level guidance summarizing effective, saturated, and underexplored strategy families; and SA uses the parent, retrieved inspirations, and landscape guidance to generate a new program together with its strategy description. After evaluation, the strategy description is embedded and the new entry is inserted into the archive.

The three modules operate at different timescales. SA runs at every strategy-guided generation. SER uses a warm-up fallback for the first C generations and switches to cluster-based retrieval once enough strategy diversity has accumulated. SLN runs every Δ generations and its guidance persists between updates. All modules operate through the context construction stage: the base framework still sample, evaluates, and selects programs in the usual way, while SEAEVO changes how mutation context is assembled and what strategic state the archive maintains.

Exploration ratio. To avoid over-constraining the search by the current strategy representation, SEAEVO uses an ε -greedy scheme: with probability ε it bypasses SA, SER, and SLN and falls back to the base framework’s default mutation pipeline; with probability $1 - \varepsilon$ it executes the full SEAEVO pipeline. This design balances strategy-guided exploitation with unguided exploration.

3.3 Strategy Articulation

Vanilla LLM mutation often leaves the search direction implicit: the model is asked to improve a program, but the failure mode and intended algorithmic change are not explicitly represented (Sharma, 2025). SA makes this reasoning explicit by asking the LLM to first articulate a strategic direction before implementing code. Given a selected parent program P_{parent} , an inspiration set $\mathcal{G}_t$, and optional landscape guidance $\mathcal{L}_t$, SA produces a new program P' and a strategy description d' using a single structured LLM prompt.

The prompt follows a *diagnose–direct–implement* format. First, the LLM diagnoses the limitations of P_{parent} using the evaluation signals available to the underlying evolutionary framework, such

as scalar fitness, execution errors, or task-specific feedback when provided. SA does not require a specific feedback format: in scalar-only settings, the diagnosis is based on the parent score, the retrieved strategy inspirations, and the task objective; when richer feedback is available, it can be included as additional context. Second, conditioned on this diagnosis, $\mathcal{G}_t$, and optional SLN guidance $\mathcal{L}_t$, the LLM proposes a concise strategy description d' for the next candidate. When $\mathcal{L}_t$ is available, it can steer the proposal away from saturated strategy families and toward underexplored alternatives. Finally, the LLM implements a new program P' following the proposed strategy description. The diagnosis is used only as intermediate prompt context. After evaluation, the P' and d' persist in the archive, with d' embedded for downstream clustering and retrieval.

This design makes strategy descriptions reusable beyond the immediate mutation step. Rather than treating reflection as transient prompt content, SA converts each proposed direction into persistent archive state that can later be clustered by SER and summarized by SLN.

3.4 Stratified Experience Retrieval

Many evolutionary frameworks construct mutation context by selecting high-fitness programs or recent ancestors and using their source code as inspiration for the next mutation (Novikov et al., 2025; Lange et al., 2025). While code-level inspiration provides useful implementation details, it is also costly and noisy: programs can span hundreds of lines, and syntactic similarity does not necessarily reflect shared algorithmic intent. Moreover, fitness-based selection can introduce a self-reinforcing bias, in which high-scoring strategy families are repeatedly surfaced while strategically distinct but temporarily lower-scoring candidates receive little exposure. SER addresses these issues by organizing the archive in strategy-embedding space and retrieving inspirations that cover both similar and distinct strategy families while remaining behaviorally complementary.

Strategy clustering. SER partitions the archive into C strategy clusters $\{S_1, \dots, S_C\} \leftarrow k\text{-means}(\{\mathbf{e}_i\}, C)$ over strategy embeddings (Lloyd, 1982), recomputed at every generation. This groups syntactically different programs that instantiate similar algorithmic ideas into the same strategy family, providing the semantic structure for retrieval. During the warm-up stage ($|\mathcal{A}| < C$), when the archive is too small for reliable clustering, SER bypasses the cluster structure and returns the parent, the global best, and the program with the highest complementary coverage relative to the global best.

Complementary Coverage Score. SER ranks candidate inspirations by how many of the reference program’s failures a candidate can solve. Given a reference program P_{ref} and a fixed validation set $\mathcal{V}$, each program P_i has a binary success vector $\mathbf{b}_i \in \{0, 1\}^{|\mathcal{V}|}$. We define the *Complementary Coverage score* (ComCov) of P_i with respect to P_{ref} is

$$\text{ComCov}(P_i; P_{\text{ref}}) = \frac{1}{|\mathcal{V}|} \sum_{k=1}^{|\mathcal{V}|} \mathbf{1}[b_{\text{ref}}^k = 0 \wedge b_i^k = 1], \quad (2)$$

i.e., the fraction of validation instances that P_{ref} fails but P_i solves. ComCov rewards candidates that cover the reference program’s failures, making retrieval complementary to the parent’s weaknesses rather than merely diverse. When only a scalar fitness s_i is available (e.g., in geometric optimization tasks where instance-level success vectors are undefined), ComCov is identically zero and ranking falls back to fitness via the lexicographic tiebreaker $(\text{ComCov}(P_i; \cdot), s_i)$.

Inspiration selection. SER anchors retrieval on the parent program P_{parent} chosen by the base framework. After warm-up ($t \geq C$), let S_{c^*} denote the cluster containing P_{parent} . SER selects two inspirations greedily to maximize marginal complementary coverage:

$$P^{\text{intra}} = \arg \max_{P_i \in S_{c^*} \setminus \{P_{\text{parent}}\}} \text{ComCov}(P_i; P_{\text{parent}}), \quad (3)$$

$$c' \sim \text{Uniform}\{c : c \neq c^*\},$$

$$P^{\text{cross}} = \arg \max_{P_i \in S_{c'}} \text{ComCov}_{\Delta}(P_i; P_{\text{parent}}, P^{\text{intra}}), \quad (4)$$

where ComCov_{Δ} counts only instances failed by both P_{parent} and P^{intra} , ensuring non-redundant coverage. The intra-cluster pick identifies a sibling that shares the parent’s strategic family but covers different failure cases; the cross-cluster pick then targets the residual failures that neither the parent nor the sibling can solve. Random cluster sampling for c' , rather than a global argmax, prevents the

cross-cluster slot from being monopolized by a single dominant family, spreading exposure across the strategy landscape over successive generations.

For the two retrieved inspirations, SER passes only their strategy descriptions and fitness scores, not their source code. This reduces the inspiration portion of the prompt by one to two orders of magnitude and forces the mutation LLM to reason at the level of algorithmic ideas rather than copy code fragments.

3.5 Strategic Landscape Navigation

SER diversifies the local mutation context, but search may still concentrate within a small number of strategy families. SLN adds a slower-timescale mechanism that summarizes the population-level strategy landscape and guides future mutations away from saturated regions.

Every Δ generations, SLN collects the strategy descriptions and fitness scores of programs in the current archive and passes them to an LLM. The LLM produces structured landscape guidance $\mathcal{L}_t$ with four components: (i) *effective directions*: strategy families showing consistent improvement and worth further exploitation; (ii) *saturated directions*: families that appear to have plateaued and should be de-emphasized; (iii) *underexplored directions*: plausible approaches not yet well represented in the archive; and (iv) *concrete guidance*: actionable suggestions for subsequent mutation steps. The resulting $\mathcal{L}_t$ is appended to the SA prompt and remains active until the next SLN update.

This design treats saturation and opportunity detection as a semantic landscape-analysis problem rather than a fixed scalar rule. By reasoning over strategy descriptions together with their fitness trends, SLN can identify families that are repeatedly refined without further improvement and suggest qualitatively different alternatives that would be difficult to recover from fitness values alone.

4 Experiments

4.1 Experiment Setup

Tasks and datasets. We evaluate on three task families: (i) four mathematical optimization tasks, including Circle Packing in Square, Circle Packing in Rectangle, Heilbronn Triangles, and MinMax Distance; (ii) four systems optimization tasks from the ADRS benchmark (Cheng et al., 2025): Prism, TXN, EPLB, and LLM-SQL; and (iii) agent-scaffold optimization tasks.

Baselines. We compare SEAEVO against state-of-the-art LLM-driven evolutionary methods, including GEPA (Agrawal et al., 2025), OpenEvolve (Sharma, 2025), and ShinkaEvolve (Lange et al., 2025). We instantiate two variants of our method: SEAEVO_{SHINKA} over ShinkaEvolve, and SEAEVO_{OPEN} over OpenEvolve; the corresponding backbone rows therefore serve as direct ablations of the strategy-aware modules. For readability, figures use the expanded form (e.g., SEAEVO + ShinkaEvolve) while tables use the compact notation (e.g., SEAEVO_{SHINKA}). We additionally apply SEAEVO to GEPA and to AdaEvolve (Cemri et al., 2026) in Appendix E to test plug-in generality.

Implementation details. We use two backbone LLMs as the mutation model within the evolutionary loop: MiMo-V2-Pro (Xiaomi MiMo Team, 2026) and Gemini-3-Flash (Google DeepMind, 2025). For the agent-scaffold task, the inference model is Qwen3-8B (Yang et al., 2025a). Strategy embeddings use text-embedding-3-small model (OpenAI, 2024). We build on the SkyDiscover framework (Liu et al., 2026c). Each configuration runs for $T = 100$ generations with SLN update interval $\Delta = 10$, $C = 5$ strategy clusters, and ε -greedy ratio $\varepsilon = 0.2$. We use the default hyperparameters for all baseline methods. Similar to Cemri et al. (2026), each experiment is repeated three times; we report mean and standard deviation. More details are provided in Appendix D.

4.2 Main Results

Tables 1 and 2 report results on mathematical and systems optimization benchmarks, respectively. Figure 3 and 6 report agent-scaffold design results, and Figure 1 shows search trajectories on the two Circle Packing benchmarks. We highlight several observations.

Consistent improvement across tasks and backbones. Across most configurations in Tables 1 and 2, SEAEVO achieves the highest average fitness, and no baseline systematically dominates on any task or backbone. The relative ranking is stable between MiMo-V2-Pro and Gemini-3-Flash,

Table 1: Performance on mathematical optimization benchmarks. Avg denotes mean score with standard deviation over 3 runs, and Best denotes the best score achieved. Best results within each backbone group are bolded, and full-precision near ties are reported in Table 7.

Method	Circle Packing (Rect)		Circle Packing (Square)		Heilbronn (Triangles)		MinMax Distance	
	Avg	Best	Avg	Best	Avg	Best	Avg	Best
<i>Backbone: MiMo-V2-Pro</i>								
GEPA	2.3398 $\pm$ 0.0197	2.3584	2.5206 $\pm$ 0.0207	2.5414	0.0265 $\pm$ 0.0010	0.0273	0.2760 $\pm$ 0.0019	0.2776
OpenEvolve	2.1380 $\pm$ 0.1987	2.3530	2.4003 $\pm$ 0.0386	2.4390	0.0250 $\pm$ 0.0035	0.0281	0.2564 $\pm$ 0.0165	0.2760
ShinkaEvolve	2.3560 $\pm$ 0.0083	2.3608	2.4812 $\pm$ 0.0442	2.5123	0.0272 $\pm$ 0.0001	0.0273	0.2776 $\pm$ 0.0008	0.2783
SEAEVO _{OPEN}	2.3607 $\pm$ 0.0036	2.3642	2.6045 $\pm$ 0.0338	2.6263	0.0314 $\pm$ 0.0014	0.0330	0.2778 $\pm$ 0.0003	0.2781
SEAEVO _{SHINKA}	2.3603 $\pm$ 0.0033	2.3640	2.5467 $\pm$ 0.1292	2.6237	0.0274 $\pm$ 0.0013	0.0285	0.2780 $\pm$ 0.0003	0.2784
<i>Backbone: Gemini-3-Flash</i>								
GEPA	2.3610 $\pm$ 0.0013	2.3621	2.5306 $\pm$ 0.0380	2.5574	0.0303 $\pm$ 0.0011	0.0316	0.2785 $\pm$ 0.0000	0.2785
OpenEvolve	2.1713 $\pm$ 0.0432	2.1968	2.4522 $\pm$ 0.0387	2.4887	0.0062 $\pm$ 0.0022	0.0087	0.2765 $\pm$ 0.0007	0.2774
ShinkaEvolve	2.3625 $\pm$ 0.0030	2.3658	2.5204 $\pm$ 0.0357	2.5617	0.0317 $\pm$ 0.0012	0.0331	0.2783 $\pm$ 0.0001	0.2785
SEAEVO _{OPEN}	2.3643 $\pm$ 0.0026	2.3658	2.5802 $\pm$ 0.0822	2.6317	0.0305 $\pm$ 0.0003	0.0309	0.2785 $\pm$ 0.0000	0.2785
SEAEVO _{SHINKA}	2.3639 $\pm$ 0.0034	2.3658	2.6297 $\pm$ 0.0077	2.6353	0.0335 $\pm$ 0.0006	0.0342	0.2785 $\pm$ 0.0000	0.2785

Table 2: Performance on systems optimization benchmarks with Gemini-3-Flash. Avg denotes mean score with standard deviation over runs, and Best denotes the best score achieved. The Human/SOTA row reports reference best scores when available. Best method results are highlighted in boldface.

Method	Prism		TXN		EPLB		LLM-SQL	
	Avg	Best	Avg	Best	Avg	Best	Avg	Best
Human/SOTA	–	21.89	–	2725	–	0.1265	–	0.692
GEPA	23.6733 $\pm$ 0.6034	24.0217	3266.0 $\pm$ 243.1	3496.5	0.1472 $\pm$ 0.0062	0.1543	0.7158 $\pm$ 0.0053	0.7233
OpenEvolve	24.0217 $\pm$ 0.0000	24.0217	3586.7 $\pm$ 188.4	3745.3	0.1260 $\pm$ 0.0002	0.1262	0.7117 $\pm$ 0.0105	0.7241
ShinkaEvolve	26.2672 $\pm$ 0.2757	26.5369	3480.7 $\pm$ 48.6	3508.8	0.1326 $\pm$ 0.0111	0.1454	0.6998 $\pm$ 0.0165	0.7181
SEAEVO _{OPEN}	26.1783 $\pm$ 0.1031	26.2783	3810.6 $\pm$ 416.2	4098.4	0.1535 $\pm$ 0.0158	0.1716	0.7104 $\pm$ 0.0010	0.7114
SEAEVO _{SHINKA}	43.6149 $\pm$ 29.7823	78.0040	3886.1 $\pm$ 517.9	4219.4	0.1958 $\pm$ 0.0002	0.1960	0.7200 $\pm$ 0.0087	0.7318

suggesting that gains stem from the strategy-aware modules rather than backbone-specific behavior. The same pattern holds on the agent-scaffold design task (see Figure 3 and 6), where SEAEVO improves both OpenEvolve and ShinkaEvolve backbones.

Gains scale with strategy-space openness. SEAEVO’s advantage is largest on systems optimization benchmarks. SEAEVO_{SHINKA} achieves a 32.1% average relative improvement in Avg score over its ShinkaEvolve backbone across the four ADRS benchmarks. On the high-variance Prism benchmark, one run discovers a substantially stronger solution, yielding a near-3 $\times$ best-score improvement over ShinkaEvolve. Improvements are more moderate on mathematical tasks (e.g., +4.3% on Circle Packing Square Avg in Table 1), and diminish on narrower-solution-space tasks such as LLM-SQL. We attribute this gradient to SLN and SER providing stronger signal when qualitatively distinct strategy families exist to be discovered and distinguished; when the strategy space is constrained, fitness-only baselines are harder to improve upon.

Faster convergence at lower cost. Figure 1 shows search trajectories on the Circle Packing benchmarks. Both SEAEVO_{OPEN} and SEAEVO_{SHINKA} reach the Human/SOTA reference earlier than their underlying backbones. On Circle Packing (Square), SEAEVO_{OPEN} crosses 2.6 within 30 generations whereas OpenEvolve plateaus near 2.47 throughout 100 generations. The cumulative-cost curves show that SEAEVO’s API cost grows slower than the backbones’, so the gains do not come from additional compute per generation; they come from reaching better solutions in fewer generations.

4.3 Ablation Study

Table 3 reports ablation results on Circle Packing (Square) with Gemini-3-Flash. All three modules improve over the no-module baseline, but with distinct roles. SA and SER individually yield modest gains in average fitness while substantially raising the best fitness, suggesting they primarily expand the reachable solution space. SLN alone produces the largest standalone improvement in average fitness and the earliest convergence, indicating that landscape-level guidance provides the strongest global search signal. The full SEAEVO achieves the best average fitness, finds its best solution earliest,

Table 3: Ablation study on Circle Packing (Square) with Gemini-3-Flash. We report mean fitness score (Avg), standard deviation (Std), best score (Best), generation at which best was found (Gen@Best), and average API cost (AvgCost) across three seeds. We highlight the best and worst values for each metric in green and red, respectively.

SA	SER	SLN	Avg	Std ↓	Best	Gen@Best ↓	AvgCost ↓
✗	✗	✗	2.5023	0.0086	2.5099	99	\$0.5367
✓	✗	✗	2.5043	0.0855	2.6310	67	\$0.6062
✗	✓	✗	2.5224	0.0731	2.6310	95	\$0.5088
✗	✗	✓	2.5634	0.0791	2.6356	61	\$0.5614
✓	✓	✗	2.5172	0.0774	2.6304	59	\$0.6576
✓	✗	✓	2.5870	0.0816	2.6308	93	\$0.7147
✗	✓	✓	2.5901	0.0724	2.6360	88	\$0.5731
✓	✓	✓	2.6297	0.0077	2.6353	44	\$0.7075

Table 4: Hyperparameter sensitivity on Circle Packing (Square) with Gemini-3-Flash. Baseline is the full SEAEVO configuration ($\Delta=10$, $k=5$, $\varepsilon=0.2$). Each row varies one hyperparameter while holding the others fixed. The best results are in boldface.

Param	Value	Avg	Std ↓	Best	Gen@Best ↓	AvgCost ↓
Δ (SLN interval)	$\Delta = 5$	2.5628	0.1195	2.6343	80	\$0.7710
	$\Delta = 10$	2.6297	0.0077	2.6353	44	\$0.7075
	$\Delta = 20$	2.5742	0.0854	2.6307	74	\$0.6802
C (clusters)	$C = 3$	2.6283	0.0052	2.6343	86	\$0.7006
	$C = 5$	2.6297	0.0077	2.6353	44	\$0.7075
	$C = 8$	2.6175	0.0173	2.6307	72	\$0.7079
ε (Exploration ratio)	$\varepsilon = 0.0$	2.6341	0.0020	2.6360	78	\$0.7711
	$\varepsilon = 0.2$	2.6297	0.0077	2.6353	44	\$0.7075
	$\varepsilon = 0.4$	2.6263	0.0112	2.6360	95	\$0.6669

and most strikingly reduces variance by an order of magnitude relative to every two-module variant, reflecting a non-additive synergy among the three modules. Notably, the SA+SER configuration underperforms either module alone in average fitness: without SLN’s landscape signal, structured mutation paired with diversity-aware retrieval can over-commit to a narrow set of directions. This pattern suggests that SLN’s role is not merely additive but corrective, it prevents the other two modules from locking onto premature strategy families.

4.4 Hyperparameter Sensitivity

Table 4 reports sensitivity to three key hyperparameters on Circle Packing (Square).

SLN update interval Δ . $\Delta = 10$ achieves the best average fitness and earliest convergence. Updating too frequently ($\Delta = 5$) destabilizes search: the landscape changes before the population has accumulated enough new candidates to make the LLM’s judgments reliable, producing the highest variance across all settings. Updating too infrequently ($\Delta = 20$) reduces cost but lets ineffective directions persist past the point where they should be abandoned, hurting average fitness.

Number of clusters C . Performance is robust around $C = 5$, with modest degradation at $C = 3$ (coarser strategy partitioning conflates distinct families) and $C = 8$ (too few programs per cluster make intra-cluster retrieval less informative). The low sensitivity across this range suggests that SEAEVO does not require careful tuning of C .

Exploration ratio ε . $\varepsilon = 0$ (fully strategy-guided, no random exploration) reaches the highest best score but does so substantially later than $\varepsilon = 0.2$, indicating that occasional fallbacks to the base mutation pipeline accelerate escape from premature strategy clusters. $\varepsilon = 0.4$ dilutes the strategy-aware pipeline too aggressively, hurting both average fitness and convergence speed. $\varepsilon = 0.2$ provides the best balance of guided exploration and stochastic diversity.

4.5 Visualization

To examine whether SEAEVO changes the distribution of explored strategies rather than merely improving the final score, we project the strategy text associated with all generated programs into two dimensions using t-SNE (Van der Maaten and Hinton, 2008). For SEAEVO_{SHINKA}, this text is

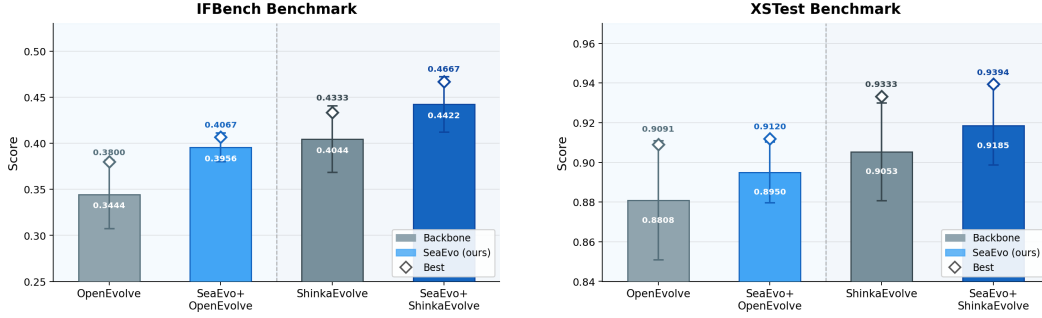


Figure 3: Performance on the agent scaffold design benchmark with maximum one LLM call. Bars show average test score with standard deviation across runs, and diamonds indicate the best score.

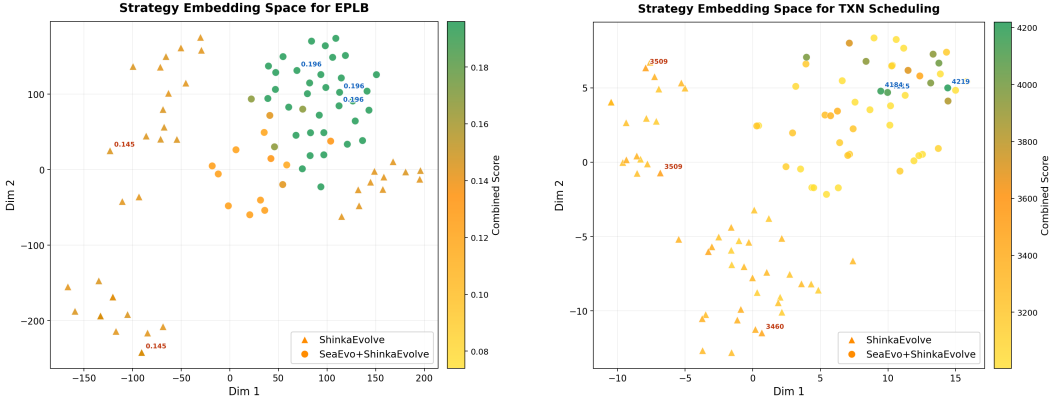


Figure 4: Strategy embedding spaces on two systems optimization benchmarks. Each point denotes a candidate program projected from its strategy embedding by t-SNE, color indicates the combined score, and marker shape distinguishes the base backbone from SEAEVO-augmented search.

the explicit strategy description stored in the archive; for the ShinkaEvolve baseline, which does not maintain such descriptions, we approximate it using docstrings and comments extracted from each generated program. Figure 4 shows the resulting qualitative projections on two representative systems optimization benchmarks, EPLB and TXN Scheduling. Each point denotes one generated program; marker shape distinguishes ShinkaEvolve from SEAEVO_{SHINKA}, and color encodes the combined score. The projections reveal a clear difference in the regions explored by the two search processes. On EPLB, ShinkaEvolve candidates concentrate in a few broad regions, whereas SEAEVO_{SHINKA} produces a dense group of high-scoring candidates in a different part of the strategy space. This suggests that the improvement is associated with discovering and refining a distinct strategy family, rather than only isolated high-scoring mutations. On TXN Scheduling, ShinkaEvolve again concentrates in a small number of regions, while the top SEAEVO_{SHINKA} candidates appear in a separate area of the projection. This is consistent with the intended role of SLN: redirecting mutations toward underexplored strategy families when local directions begin to saturate.

5 Conclusion

We introduced SEAEVO, a persistent strategy-space layer that turns language-level strategic reasoning into population-level evolutionary state. By associating each candidate program with a natural-language strategy description, clustering the archive by strategy semantics, retrieving behaviorally complementary inspirations, and monitoring strategy-family saturation, SEAEVO transforms the archive from a flat record of programs into a navigable map of algorithmic ideas. Across mathematical algorithm discovery, systems optimization, and agent-scaffold benchmarks, SEAEVO improves existing evolutionary backbones in most settings, with the largest gains on open-ended tasks. It also improves the search frontier by reaching competitive solutions in fewer generations and at lower cumulative API cost. These results point toward LLM-guided evolutionary systems that improve over time by accumulating, organizing, and reusing algorithmic strategies.

References

- Agrawal, L. A., Tan, S., Soylu, D., Ziemis, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., and Khattab, O. (2025). GEPA: reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*.
- Assumpção, H., Ferreira, D., Campos, L., and Murai, F. (2025). Codeevolve: An open source evolutionary coding agent for algorithm discovery and optimization. *arXiv preprint arXiv:2510.14150*.
- Cemri, M., Agrawal, S., Gupta, A., Liu, S., Cheng, A., Mang, Q., Naren, A., Erdogan, L. E., Sen, K., Zaharia, M., Dimakis, A., and Stoica, I. (2026). Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*.
- Chen, C., Zhong, M., Fan, Y., Shi, J., and Sun, J. (2025). Hifo-prompt: Prompting with hindsight and foresight for llm-based automatic heuristic design. *arXiv preprint arXiv:2508.13333*.
- Cheng, A., Liu, S., Pan, M. Z., Li, Z., Wang, B., Krentsel, A., Xia, T., Cemri, M., Park, J., Yang, S., Chen, J., Agrawal, L. A., Desai, A., Xing, J., Sen, K., Zaharia, M., and Stoica, I. (2025). Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*.
- Google DeepMind (2025). Gemini 3 flash. <https://deepmind.google/models/gemini/flash/>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.
- Lange, R. T., Imajuku, Y., and Cetin, E. (2025). Shinkaevolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*.
- Lee, Y., Boen, J., and Finn, C. (2025). Feedback descent: Open-ended text optimization via pairwise comparison. *arXiv preprint arXiv:2511.07919*.
- Li, T., Wang, Y., Chen, Z., Wang, Z., Ma, L., and Qi, G.-j. (2025). C-evolve: Consensus-based evolution for prompt groups. *arXiv preprint arXiv:2509.23331*.
- Liu, F., Xialiang, T., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. (2024). Evolution of heuristics: Towards efficient automatic algorithm design using large language model. In *International Conference on Machine Learning*, pages 32201–32223. PMLR.
- Liu, F., Yao, Y., Guo, P., Yang, Z., Lin, X., Zhao, Z., Tong, X., Mao, K., Lu, Z., Wang, Z., and Zhang, Q. (2026a). A systematic survey on large language models for algorithm design. *ACM Computing Surveys*, 58(8):1–32.
- Liu, F., Yi, H., Luo, S., Wang, Y., Yang, Y., Li, X., Hu, Z., Feng, J., and Liu, Q. (2025). Cognitive alpha mining via llm-driven code-based evolution. *arXiv preprint arXiv:2511.18850*.
- Liu, S., Agarwal, S., Maheswaran, M., Cemri, M., Li, Z., Mang, Q., Naren, A., Boneh, E., Cheng, A., Pan, M. Z., Du, A., Keutzer, K., Dimakis, A. G., Sen, K., Zaharia, M., and Stoica, I. (2026b). Evox: Meta-evolution for automated discovery. *CoRR*, abs/2602.23413.
- Liu, S., Cemri, M., Agarwal, S., Krentsel, A., Naren, A., Mang, Q., Li, Z., Gupta, A., Maheswaran, M., Cheng, A., Pan, M., Boneh, E., Ramchandran, K., Sen, K., Dimakis, A. G., Zaharia, M., and Stoica, I. (2026c). Skydiscover: A flexible framework for ai-driven scientific and algorithmic discovery.
- Lloyd, S. (1982). Least squares quantization in pcm. *IEEE transactions on information theory*, 28(2):129–137.
- Mouret, J.-B. and Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*.

- Novikov, A., Vu, N., Eisenberger, M., Dupont, E., Huang, P., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J. R., Mehrabian, A., Kumar, M. P., See, A., Chaudhuri, S., Holland, G., Davies, A., Nowozin, S., Kohli, P., and Balog, M. (2025). Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*.
- OpenAI (2024). New embedding models and API updates. <https://openai.com/blog/new-embedding-models-and-api-updates>.
- Pyatkin, V., Malik, S., Graf, V., Ivison, H., Huang, S., Dasigi, P., Lambert, N., and Hajishirzi, H. (2025). Generalizing verifiable instruction following. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J. R., Ellenberg, J. S., Wang, P., Fawzi, O., Kohli, P., and Fawzi, A. (2024). Mathematical discoveries from program search with large language models. *Nat.*, 625(7995):468–475.
- Röttger, P., Kirk, H., Vidgen, B., Attanasio, G., Bianchi, F., and Hovy, D. (2024). Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5377–5400.
- Sharma, A. (2025). Openevolve: an open-source evolutionary coding agent.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-sne. *Journal of machine learning research*, 9(11).
- Van Stein, N. and Bäck, T. (2024). Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE Transactions on Evolutionary Computation*, 29(2):331–345.
- Wu, X., Wang, D., Wu, C., Wen, L., Miao, C., Xiao, Y., and Zhou, Y. (2025). Efficient heuristics generation for solving combinatorial optimization problems using large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 3228–3239.
- Xiaomi MiMo Team (2026). MiMo-V2-Pro. <https://mimo.xiaomi.com/mimo-v2-pro>.
- Yan, M., Peng, B., Coleman, B., Chen, Z., Xie, Z., Chen, S., He, Z., Sachdeva, N., Ye, I., Wang, W., Wang, C., Chi, E. H., Pereira, F., Kang, W., Cheng, D. Z., and Wang, B. (2026). Pacevolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. (2025a). Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Yang, X., Zhang, L., Qian, H., Song, L., and Bian, J. (2025b). Heuragenix: Leveraging llms for solving complex combinatorial optimization challenges. *arXiv preprint arXiv:2506.15196*.
- Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. (2024). Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608.
- Yuksekgonul, M., Bianchi, F., Boen, J., Liu, S., Lu, P., Huang, Z., Guestrin, C., and Zou, J. (2025). Optimizing generative ai by backpropagating language model feedback. *Nature*, 639(8055):609–616.

A Limitations

We identify several limitations of this work. First, SEAEVO depends on LLMs at multiple stages: generating strategy descriptions in Strategy Articulation, judging which strategy families are saturated or underexplored in Strategic Landscape Navigation, and producing meaningful embeddings for clustering in Stratified Experience Retrieval. If the backbone LLM lacks sufficient domain knowledge, it may mischaracterize promising directions, prematurely abandon useful strategy families, or fail to redirect search away from exhausted ones, which can degrade performance. Second, due to resource constraints, we evaluate SEAEVO with two backbone LLMs across three task families, with each configuration repeated three times. Although the plug-in experiments with GEPA and AdaEvolve provide additional evidence of generality, our experiments do not cover the full range of domains in which LLM-driven evolution may be applied. Third, SEAEVO is most effective when qualitatively distinct strategy families exist and can be meaningfully described in natural language. For tasks with narrow strategy spaces, where the algorithmic design space is highly constrained, natural-language strategy descriptions may fail to capture the relevant axes of variation, limiting the benefit of strategy-space organization.

B Broader Impact

This work aims to improve LLM-guided evolutionary search by organizing search around persistent natural-language strategy representations. SEAEVO may reduce the manual effort required to design optimization heuristics and accelerate scientific, engineering, and systems-optimization workflows where reliable programmatic evaluators are available. It may also make search processes more interpretable by exposing which strategy families have been explored, saturated, or underexplored.

More capable automated program-search systems may also introduce risks if used without appropriate validation. Generated programs can be incorrect, inefficient, brittle, or unsafe outside the benchmark environments in which they are evaluated. In infrastructure, resource-allocation, or safety-critical domains, deploying automatically discovered algorithms without human review could lead to harmful outcomes.

We mitigate these risks in our experiments through controlled benchmark environments, deterministic evaluators where possible, and sandboxed execution. For practical deployment, we recommend task-specific safety checks, resource limits, robustness testing, and human review before using automatically discovered algorithms in production or safety-critical settings.

C Algorithm

Algorithm 1 presents the complete SEAEVO procedure. The outer loop iterates for T generations. At each step, SLN refreshes landscape guidance when scheduled (line 5); SER assembles a strategy-diverse, behaviorally complementary context (line 11); and SA generates a new program together with its strategy description (line 12). After evaluation, the strategy-augmented archive is updated using the base framework’s archive update rule (line 15). Here, $\mathbf{b}_i$ denotes optional instance-level outcomes when available; in scalar-only tasks it can be omitted or treated as empty.

D Experiment Setup

D.1 Datasets and Tasks

For mathematical optimization, we evaluate on four algorithm discovery tasks: Circle Packing (Rectangle), Circle Packing (Square), Heilbronn Triangles, and MinMax Distance. For systems optimization, we use four tasks from the ADRS benchmark (Cheng et al., 2025): Prism, TXN, EPLB, and LLM-SQL. The detailed task descriptions and prompts are provided in Appendix G.

For agent-scaffold design, we use the IFBench (Pyatkin et al., 2025) and XSTest benchmark (Röttger et al., 2024). We evaluate each benchmark under two inference-budget settings, where the inference model is allowed to make either one or up to three LLM calls per example. We split the benchmark into a feedback set, a metric set, and a test set, with 60, 75, and 150 examples, respectively. The

Algorithm 1 SEAEVO: Strategy-Space Evolution

Require: Initial program P_0 , total generations T , SLN interval Δ , number of clusters C , exploration ratio ε
Ensure: Best program P^*

- 1: Evaluate P_0 to obtain s_0 and optional $\mathbf{b}_0$; $d_0 \leftarrow \text{LLM}_{\text{describe}}(P_0)$; $\mathbf{e}_0 \leftarrow \text{embed}(d_0)$
- 2: $\mathcal{A} \leftarrow \{(P_0, s_0, d_0, \mathbf{e}_0, \mathbf{b}_0)\}$; $P^* \leftarrow P_0$; $s^* \leftarrow s_0$; $\mathcal{L} \leftarrow \emptyset$
- 3: **for** $t = 1, 2, \dots, T$ **do**
- 4: $P_{\text{parent}} \leftarrow \text{BASESELECT}(\mathcal{A})$
- 5: **if** $t \bmod \Delta = 0$ **then**
- 6: $\mathcal{L} \leftarrow \text{SLN}(\mathcal{A})$ $\triangleright$ refresh landscape guidance; persists until next update
- 7: **end if**
- 8: **if** $\text{rand}() < \varepsilon$ **then** $\triangleright \varepsilon$ -greedy fallback
- 9: $P' \leftarrow \text{BASEMUTATE}(P_{\text{parent}}, \mathcal{A})$; $d' \leftarrow \text{LLM}_{\text{describe}}(P')$
- 10: **else**
- 11: $\mathcal{G}_t \leftarrow \text{SER}(\mathcal{A}, P_{\text{parent}}, C)$
- 12: $(P', d') \leftarrow \text{SA}(P_{\text{parent}}, \mathcal{G}_t, \mathcal{L})$
- 13: **end if**
- 14: Evaluate P' to obtain s' and optional $\mathbf{b}'$; $\mathbf{e}' \leftarrow \text{embed}(d')$
- 15: $\mathcal{A} \leftarrow \text{BASEUPDATE}(\mathcal{A}, (P', s', d', \mathbf{e}', \mathbf{b}'))$
- 16: **if** $s' > s^*$ **then**
- 17: $P^* \leftarrow P'$; $s^* \leftarrow s'$
- 18: **end if**
- 19: **end for**
- 20: **return** P^*

Algorithm 2 SA: Strategy Articulation

Require: Parent program P_{parent} , inspiration set $\mathcal{G}_t$, landscape guidance $\mathcal{L}$
Ensure: New program P' , strategy description d'

- 1: $(\text{diag}, d', P') \leftarrow \text{LLM}_{\text{SA}}(P_{\text{parent}}, \mathcal{G}_t, \mathcal{L})$ $\triangleright$ single call: diagnose $\rightarrow$ direct $\rightarrow$ implement
- 2: **return** P', d'

feedback set is used during evolution, the metric set is used for candidate selection, and the test set is held out for final evaluation.

D.2 Models

For evolution, we adopt two frontier models as representative non-reasoning and reasoning backbones: Gemini-3-Flash (Google DeepMind, 2025), accessed via the OpenRouter API¹, and MiMo-V2-Pro (Xiaomi MiMo Team, 2026), accessed via its official API². The decoding temperature for evolution model is set to 0.7. For agent-scaffold tasks, we use Qwen3-8B (Yang et al., 2025a) as the inference model, deployed locally with vLLM (Kwon et al., 2023) on $2 \times \text{A6000}$ GPUs. Following Li et al. (2025), the default maximum generation length is set to 4096 and the decoding temperature for the inference model is set to 1.0. The decoding temperature can be modified by the evolution model.

We compare SEAEVO against several representative LLM-driven evolutionary methods and use the corresponding open-source backbones to instantiate SEAEVO. GEPA (Agrawal et al., 2025) performs reflective prompt evolution using verbal feedback over execution traces. OpenEvolve (Sharma, 2025) is an open-source reimplementation of the AlphaEvolve (Novikov et al., 2025) paradigm. ShinkaEvolve (Lange et al., 2025) improves sample efficiency via adaptive parent sampling and novelty-based rejection. AlphaEvolve (Novikov et al., 2025) is closed-source and is included only as a reference target. We use default hyperparameters as reported in the respective papers. For OpenEvolve, we set `num_context_programs=5`, `num_islands=5`, `population_size=40`, `archive_size=100`, `exploration_ratio=0.2`, `exploitation_ratio=0.7`, `migration_interval=10`, `migration_rate=0.1`, and `feature_bins=10`. For ShinkaEvolve, we set `num_islands=3`, `archive_size=20`, `migration_interval=5`, and `migration_rate=0.2`.

Algorithm 3 SER: Stratified Experience Retrieval

Require: Archive $\mathcal{A}$, parent P_{parent} , number of clusters C **Ensure:** Inspiration context $\mathcal{G}_t$

```
1:  $P_{\text{best}} \leftarrow \arg \max_{P_i \in \mathcal{A}} s_i$ 
2: if  $|\mathcal{A}| < C$  then ▷ warm-up fallback before reliable clustering
3:    $\mathcal{B} \leftarrow \mathcal{A} \setminus \{P_{\text{parent}}, P_{\text{best}}\}$ 
4:   if  $\mathcal{B} \neq \emptyset$  then
5:      $P^{\text{div}} \leftarrow \arg \max_{P_i \in \mathcal{B}} \text{score}(P_i; P_{\text{best}})$ 
6:     return unique entries from  $\{P_{\text{parent}}, P_{\text{best}}, P^{\text{div}}\}$ 
7:   else
8:     return unique entries from  $\{P_{\text{parent}}, P_{\text{best}}\}$ 
9:   end if
10: end if
11:  $\{S_c\}_{c=1}^C \leftarrow \text{KMEANS}(\{\mathbf{e}_i : (P_i, s_i, d_i, \mathbf{e}_i, \mathbf{b}_i) \in \mathcal{A}\}, C)$ 
12:  $c^* \leftarrow$  cluster containing  $P_{\text{parent}}$ 
13: if  $S_{c^*} \setminus \{P_{\text{parent}}\} = \emptyset$  then
14:    $P^{\text{intra}} \leftarrow \arg \max_{P_i \in \mathcal{A} \setminus \{P_{\text{parent}}\}} \text{score}(P_i; P_{\text{parent}})$ 
15: else
16:    $P^{\text{intra}} \leftarrow \arg \max_{P_i \in S_{c^*} \setminus \{P_{\text{parent}}\}} \text{score}(P_i; P_{\text{parent}})$ 
17: end if
18:  $c' \sim \text{Uniform}\{c : c \neq c^*\}$  ▷ random cross-cluster sampling
19:  $P^{\text{cross}} \leftarrow \arg \max_{P_i \in S_{c'}} \text{score}_{\Delta}(P_i; P_{\text{parent}}, P^{\text{intra}})$ 
20: return  $\{P_{\text{parent}}, P^{\text{intra}}, P^{\text{cross}}\}$ 
```

Algorithm 4 SLN: Strategic Landscape Navigation

Require: Archive $\mathcal{A}$ **Ensure:** Landscape guidance $\mathcal{L}$

```
1:  $\mathcal{D} \leftarrow \{(d_i, s_i) : (P_i, s_i, d_i, \mathbf{e}_i, \mathbf{b}_i) \in \mathcal{A}\}$  ▷ collect strategy descriptions and fitness scores
2:  $\mathcal{L} \leftarrow \text{LLM}_{\text{navigate}}(\mathcal{D})$  ▷ identify effective, saturated, and underexplored families
3: return  $\mathcal{L}$ 
```

E Additional Experimental Results

E.1 Generality across additional evolutionary frameworks.

To examine whether SEAEVO is tied to a specific evolutionary backbone, we additionally integrate it into two further LLM-driven frameworks: GEPA (Agrawal et al., 2025) and AdaEvolve (Cemri et al., 2026). Table 5 shows that SEAEVO can be plugged into both backbones with no architectural changes and improves their average score on most benchmarks; the gains on AdaEvolve are particularly large on the open-ended systems tasks (TXN and EPLB), consistent with the strategy-space-openness pattern observed in the main results. The one exception is EPLB under GEPA, where SEAEVO slightly underperforms the backbone in Avg but improves Best—suggesting that GEPA’s reflective prompt evolution may already saturate parts of the strategy signal SEAEVO would otherwise add. Overall, these results indicate that SEAEVO acts as a plug-and-play strategy-space layer that generalizes beyond the OpenEvolve and ShinkaEvolve backbones used in our main experiments.

E.2 Ablation Study on TXN

Table 6 shows that the full SEAEVO framework achieves the strongest overall performance, obtaining the best average fitness and matching the highest best score among all variants. This indicates that the three modules are complementary: strategy articulation provides more deliberate mutations, stratified retrieval improves the diversity of useful inspirations, and landscape navigation helps steer the search away from over-explored directions.

Among the individual modules, SLN is the most effective standalone component, nearly matching the full model in average fitness. This suggests that strategy-level landscape guidance is particularly

¹<https://openrouter.ai/api/v1>²<https://api.xiaomimimo.com/v1>

Table 5: Generality of SEAEVO as a plug-in over additional evolutionary backbones GEPA and AdaEvolve, evaluated on a mix of mathematical and systems optimization benchmarks. Within each backbone group, SEAEVO-augmented rows (highlighted) are compared against the corresponding base framework. Best results are in boldface.

Method	Circle Packing (Rect)		Heilbronn (Triangles)		TXN		EPLB	
	Avg	Best	Avg	Best	Avg	Best	Avg	Best
GEPA	2.3610 $\pm$ 0.0013	2.3621	0.0303 $\pm$ 0.0011	0.0316	3266.0 $\pm$ 243.1	3496.5	0.1472 $\pm$ 0.0062	0.1543
+ SEAEVO	2.5408 $\pm$ 0.0442	2.6022	0.0322 $\pm$ 0.0017	0.0337	3386.0 $\pm$ 11.0	3401.0	0.1397 $\pm$ 0.0178	0.1649
AdaEvolve	2.6223 $\pm$ 0.0086	2.6285	0.0322 $\pm$ 0.0003	0.0324	3622.0 $\pm$ 305.0	4032.0	0.1270 $\pm$ 0.0002	0.1273
+ SEAEVO	2.6283 $\pm$ 0.0097	2.6360	0.0329 $\pm$ 0.0014	0.0338	4238.0 $\pm$ 598.0	5076.0	0.1597 $\pm$ 0.0111	0.1689

Table 6: Ablation study on TXN with Gemini-3-Flash. We report mean fitness (Avg), standard deviation (Std), best score (Best), generation at which best was found (Gen@Best), and average API cost (AvgCost) across three runs. We highlight the best and worst values for each metric in green and red, respectively.

SA	SER	SLN	Avg	Std ↓	Best	Gen@Best ↓	AvgCost ↓
×	×	×	3541.4	163.5	3676.5	53	0.5055
✓	×	×	3668.1	136.9	3831.4	94	0.6115
×	✓	×	3599.8	206.0	3891.1	97	0.4811
×	×	✓	3881.1	142.9	4081.6	94	0.5605
✓	✓	×	3653.4	170.4	3787.9	85	0.6205
✓	×	✓	3784.9	317.0	4219.4	89	0.6884
×	✓	✓	3737.7	351.4	4132.2	30	0.5711
✓	✓	✓	3886.1	517.9	4219.4	59	0.6966

important for TXN, where the search can otherwise spend many generations refining already saturated directions. In contrast, SA and SER alone bring more limited gains. SA improves the stability of mutation by making the intended improvement explicit, while SER mainly broadens the inspiration context. However, neither is sufficient on its own to consistently guide the global search.

The interaction between modules reveals a trade-off between early discovery and overall performance. SER+SLN finds a strong solution the earliest, indicating that complementary retrieval can accelerate landscape-guided exploration. However, the full model achieves the best overall performance, suggesting that SA provides additional mutation-level guidance that improves the quality of the search. This comes with higher variance and cost, reflecting a more aggressive and exploratory search process. Overall, these results show that SEAEVO’s advantage does not come from a single component in isolation, but from coupling local mutation-level reasoning with archive-level retrieval and landscape-level search control.

E.3 Cost Analysis

Figure 5 reports the per-run API cost breakdown on Circle Packing (Rect) and EPLB. Across both tasks and both evolutionary backbones, SEAEVO consistently reduces total cost, with savings ranging from 18.4% to 41.5%. The reduction is most pronounced on EPLB, where candidate programs are longer and code-level context becomes substantially more expensive.

The main source of savings is lower input-token cost. Instead of passing full source code for retrieved inspiration programs, SER provides compact strategy descriptions together with fitness information. This replacement substantially shortens the mutation prompt, especially for tasks with longer programs. Output cost remains comparable, and can slightly increase because SA asks the model to produce structured diagnose–direct–implement reasoning before generating the new candidate.

The additional overhead introduced by strategy embeddings and SLN is small. Embedding calls are inexpensive, and SLN is invoked only periodically, so these components do not offset the savings from shorter prompts. Overall, the results suggest that strategy-level context is not only more informative for search, but also more token-efficient than code-level context. This advantage becomes more important as the evolved programs grow longer.

Table 7: Full-precision values for results that appear tied after rounding in Table 1, including Circle Packing (Rectangle, $n=21$) and MinMax Distance.

Task	Metric	Backbone	Method	Exact Number
Circle Packing (Rect)	Best	Gemini-3-Flash	ShinkaEvolve	2.365832354910838
			SEAEVO _{OPEN}	2.365832376111432
			SEAEVO _{SHINKA}	2.365832393877214
MinMax Distance	Best	MiMo-V2-Pro	SEAEVO _{OPEN}	0.278493639030704
		Gemini-3-Flash	GEPA	0.278539322443734
			ShinkaEvolve	0.278539278338067
			SEAEVO _{OPEN}	0.278539322443734
			SEAEVO _{SHINKA}	0.278539322443734
	Mean	Gemini-3-Flash	GEPA	0.278536175170671
			SEAEVO _{OPEN}	0.278539322443661
			SEAEVO _{SHINKA}	0.278539322443734

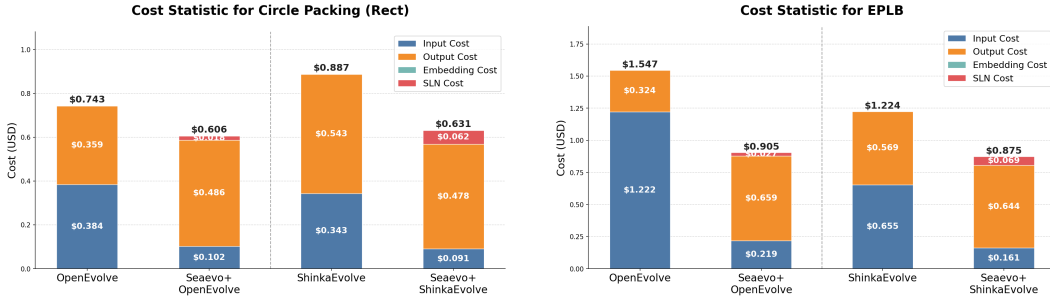


Figure 5: Cost breakdown on Circle Packing (Rect) and EPLB with Gemini-3-Flash. Bars are decomposed into input cost (prompt tokens), output cost (completion tokens), embedding cost (text-embedding-3-small calls for strategy descriptions), and SLN cost (landscape navigation for strategy navigation). SEAEVO reduces total cost in all four comparisons despite adding embedding and SLN overhead, primarily by replacing source-code context with compact strategy descriptions in the mutation prompt.

E.4 Additional Prism Runs

Prism exhibits substantially higher variance than the other ADRS benchmarks. To better characterize this behavior, we ran three additional independent SEAEVO_{SHINKA} trials on Prism. The results are shown in Table 8. Across these additional runs, SEAEVO_{SHINKA} obtains an average score of 32.99 with a sample standard deviation of 8.79.

These results confirm that Prism is a high-variance discovery task. Two of the three additional runs remain near the local plateau around 26–30, while one run escapes this region and reaches 42.94. This pattern suggests that SEAEVO’s advantage on Prism is not a uniform improvement across every seed, but rather an increased probability of discovering substantially stronger strategy families.

This behavior is consistent with the qualitative search trajectories observed in our case study. In many runs, evolution initially converges to local memory-layout or low-level transformation strategies. In successful runs, however, the strategy archive enables the search to shift toward qualitatively different restructuring strategies, resulting in large non-smooth performance jumps. Therefore, for Prism, we report both aggregate statistics and best-run behavior, since the benchmark stresses rare strategy discovery rather than smooth incremental optimization.

E.5 High-Precision Score Reference

Table 7 reports full-precision fitness scores for the Circle Packing (Rectangle) and MinMax Distance benchmarks, where multiple methods converge to values that differ only beyond the fourth decimal

Table 8: Additional Prism results over three independent runs.

	R1	R2	R3	Mean $\pm$ Std
Prism Score	26.26	42.94	29.77	32.99 $\pm$ 8.79

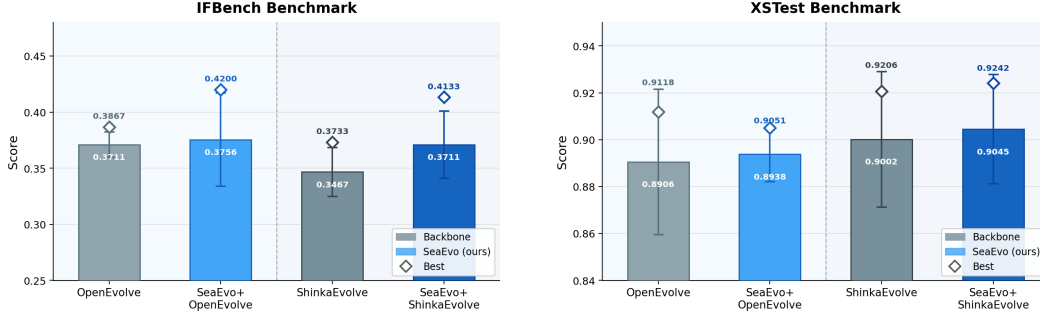


Figure 6: Performance on the agent scaffold design benchmark with maximum three LLM calls. Bars show average test score with standard deviation across runs, and diamonds indicate the best score.

place. These scores are reported to full floating-point precision to facilitate exact comparison with published reference values.

F Further Analysis

F.1 Case Study for Prism

For the first 60 generations, all discovered programs converged to a plateau near a combined score of 26. These programs shared a common structural pattern: greedy initialization followed by simulated annealing with single-model-move or pairwise-swap neighborhood operators. Despite diverse surface-level variations—different sorting criteria, cooling schedules, and tie-breaking heuristics—they were structurally trapped in the same local optimum.

SLN identifies the structural deadlock. At generation 50, the SLN diagnosed a recurring failure mode it termed "memory-locking": when a bottleneck GPU is overloaded, migrating its highest-pressure models is blocked because neighboring GPUs are also near capacity. Single-model moves and 1-for-1 swaps are insufficient to resolve this deadlock, as they cannot simultaneously free memory on one GPU while absorbing load on another. By generation 60, SLN had explicitly recommended abandoning simulated annealing in favor of Targeted Large Neighborhood Search (LNS) with a "Destroy and Repair" mechanism—specifically removing 10–20% of models from the top pressured GPUs and re-inserting them via a randomized regret heuristic.

SA translates the landscape insight into a precise diagnosis. At generation 64, SEAEVO’s SA component, operating with the SLN guidance embedded in its context, produced a diagnosis that precisely named the failure mode: "The primary bottleneck is 'memory-locking,' where a high-pressure GPU cannot offload models because other GPUs are nearly full, requiring complex multi-way swaps or 'room-clearing' moves that simple 1-for-1 swaps cannot achieve." The proposed strategy was a Targeted LNS with a "Pressure-Relief" re-insertion heuristic: identify the top-2 bottleneck GPUs, remove models with the highest pressure-to-size ratio, and repair using a regret-style insertion that prioritizes increasing minimum remaining memory cluster-wide. This directly targets the denominator of the KVPR equation to create the "movement room" required for complex re-arrangements.

The resulting algorithm achieves a 3× improvement. The implementation fully abandoned the SA loop and replaced it with a timed LNS cycle running within the 1-second wall-clock budget. The outcome was a combined score of 78.0 (max_kvpr $\approx$ 0.013), compared to the 26 plateau (max_kvpr $\approx$ 0.038). This represents not an incremental refinement but a structural algorithmic transition—from stochastic local search to targeted destroy-and-repair metaheuristics—driven by the landscape understanding accumulated by SLN over 60 generations.

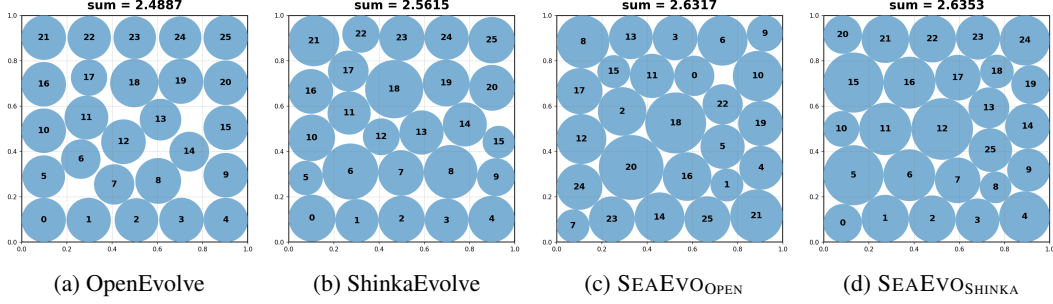


Figure 7: Gemini-3-Flash best configurations comparisons for the Circle Packing (Square) experiment.

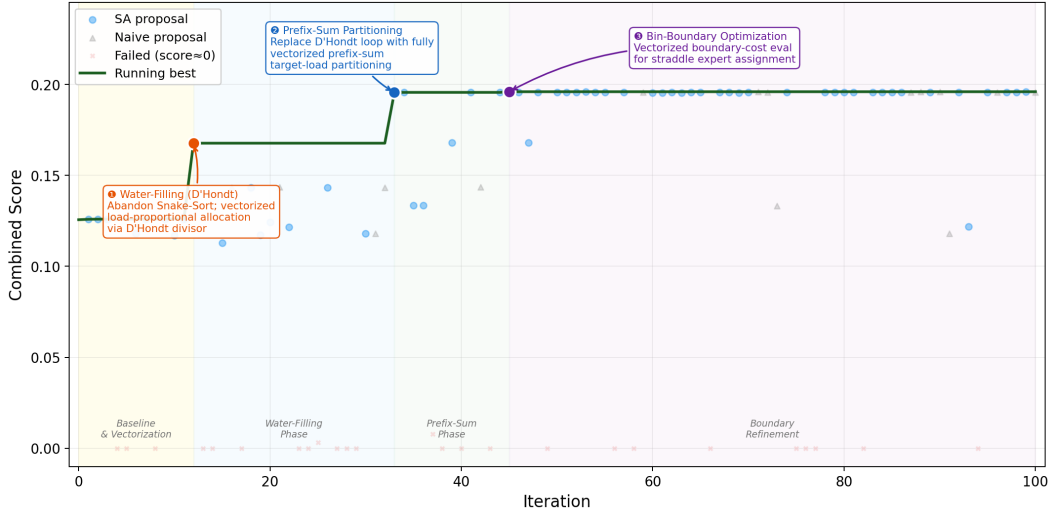


Figure 8: Strategy evolution timeline for EPLB with SEAEVO_{SHINKA}

Post-breakthrough variance. Notably, subsequent generations (65–100) failed to maintain the 78-level performance, reverting to scores of 25–26. This variance reflects a characteristic of combinatorial placement problems: the LNS solution’s quality is sensitive to random seed and the specific destroy-and-repair sequence. generations 65 onward explored modifications to the LNS implementation but did not rediscover the precise configuration that yielded the score of 78. This suggests that while SEAEVO successfully identified the correct algorithmic direction (LNS), fully exploiting this direction may require additional search budget or systematic hyperparameter tuning within the LNS framework. A strategy evolution timeline is shown in Figure 9 and the best program discovered is listed in Figure 12.

F.2 Case Study for EPLB

For the first 11 generations, all discovered programs shared a common focus: replacing the reference implementation’s sequential greedy packing with vectorized alternatives—Longest Processing Time (LPT) sorting, zig-zag batch allocation, and bulk replication via tensor operations. Despite diverse surface-level variations, these programs converged to a plateau near a combined score of ~ 0.126 . The underlying bottleneck was not execution speed but algorithmic: all variants retained the “snake-sort” packing heuristic, which assigns physical experts to GPUs in a fixed zigzag pattern and fails to minimize load variance.

SLN identifies the structural bottleneck. By generation 10, the SLN had diagnosed a recurring failure mode: the snake-sort packing introduces systematic imbalance because it does not account for heterogeneous expert loads. SLN explicitly recommended abandoning snake-sort in favor of a load-aware replication strategy. Specifically, a proportional allocation method that distributes replicas

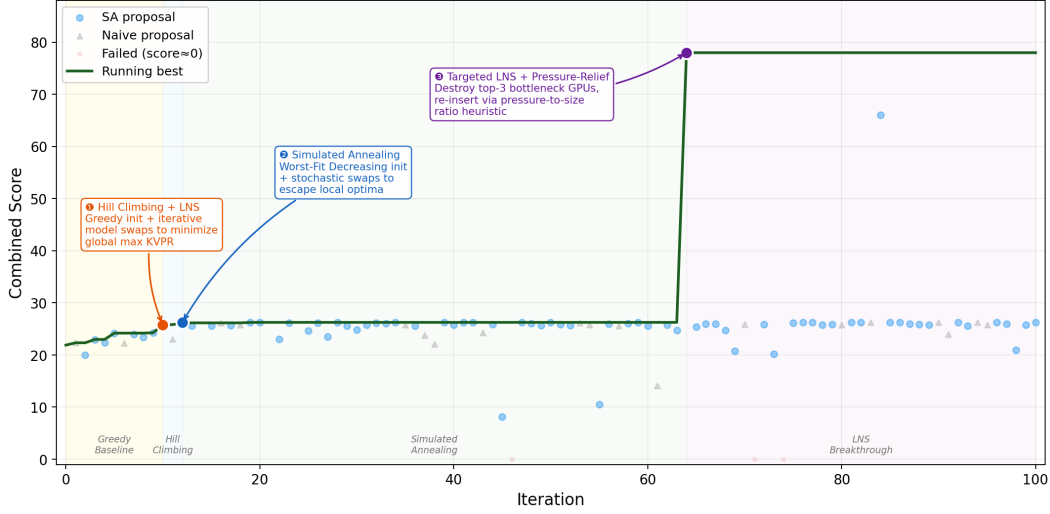


Figure 9: Strategy evolution timeline for Prism with SAEVO_{SHINKA}

in proportion to per-expert token load, combined with a globally-sorted packing that minimizes the maximum GPU load.

SAEVO translates the insight into a concrete algorithm. At generation 12, SAEVO’s SA component, operating with the SLN guidance in context, diagnosed the precise failure: "The replication logic uses a sequential loop with GPU-CPU synchronization patterns, and the snake packing heuristic fails to minimize load variance." The proposed strategy was a vectorized *Water-Filling* allocation using the D’Hondt (highest-averages) method to determine replication counts in a single pass, combined with a First-Fit Decreasing packer using cumulative-sum thresholds. This directly targets the load-variance root cause. The result was a combined score of 0.1677, a 33% improvement over the ~0.126 plateau.

Continued refinement through vectorization. Subsequent generations refined the D’Hondt approach further. At generation 33, a fully vectorized prefix-sum-based "Target Load" partitioner replaced the remaining Python loops, achieving 0.1956. At generation 45, a Bin-Boundary Optimization replaced the midpoint heuristic with precise boundary-crossing logic that explicitly minimizes maximum GPU load for "straddle experts," reaching the final score of 0.1960. Unlike the Prism case where a single structural transition produced a $3 \times$ jump, the EPLB trajectory reflects progressive algorithmic refinement, each breakthrough eliminates one remaining approximation in the vectorization pipeline, with diminishing but consistent returns. A strategy evolution timeline is shown in Figure 8 and the best program discovered is listed in Figure 13.

G Tasks and Prompts

CIRCLE PACKING IN A UNIT SQUARE

Task. Pack $n = 26$ non-overlapping circles in a unit square to maximize the sum of their radii. **AlphaEvolve target:** $\sum r_i = 2.635$

System Prompt.

You are an expert mathematician specializing in circle packing problems and computational geometry. Your task is to improve a constructor function that directly produces a specific arrangement of 26 circles in a unit square, maximizing the sum of their radii. The AlphaEvolve paper achieved a sum of 2.635 for $n=26$.

Key geometric insights: - Circle packings often follow hexagonal patterns in the densest regions - Maximum density for infinite circle packing is $\pi/(2\sqrt{3}) = 0.9069$ - Edge effects make square container packing harder than infinite packing - Circles can be placed in layers or shells when confined to a square - Similar radius circles often form regular patterns,

while varied radii allow better space utilization - Perfect symmetry may not yield the optimal packing due to edge effects
Focus on designing an explicit constructor that places each circle in a specific position, rather than an iterative search algorithm.

CIRCLE PACKING IN A RECTANGLE

Task. Pack $n = 21$ non-overlapping circles in a rectangle with perimeter 4 ($w + h = 2$, aspect ratio free) to maximize the sum of their radii. **AlphaEvolve target:** $\sum r_i = 2.3658$

System Prompt.

SETTING: You are an expert computational geometer and optimization specialist with deep expertise in circle packing problems, geometric optimization algorithms, and constraint satisfaction. Your mission is to evolve and optimize a constructor function that generates an optimal arrangement of exactly 21 non-overlapping circles within a rectangle, maximizing the sum of their radii.

PROBLEM CONTEXT: - Objective: Create a function that returns optimal (x, y, radius) coordinates for 21 circles - Benchmark: Beat the AlphaEvolve state-of-the-art result of $\text{sum_radii} = 2.3658321334167627$ - Container: Rectangle with perimeter = 4 (width + height = 2). You may choose optimal width/height ratio - Constraints: * All circles must be fully contained within rectangle boundaries * No circle overlaps (distance between centers $\geq$ sum of their radii) * Exactly 21 circles required * All radii must be positive

PERFORMANCE METRICS: 1. sum_radii : Total sum of all 21 circle radii (PRIMARY OBJECTIVE - maximize) 2. combined_score : $\text{sum_radii} / 2.3658321334167627$ (progress toward beating benchmark) 3. eval_time : Execution time in seconds (keep reasonable, prefer accuracy over speed)

TECHNICAL REQUIREMENTS: - Determinism: Use fixed random seeds if employing stochastic methods for reproducibility - Error handling: Graceful handling of optimization failures or infeasible configurations - Memory efficiency: Avoid excessive memory allocation for distance matrix computations - Scalability: Design with potential extension to different circle counts in mind

HEILBRONN TRIANGLE PROBLEM

Task. Place $n = 11$ points in or on the boundary of an equilateral triangle with vertices $(0, 0)$, $(1, 0)$, $(0.5, \sqrt{3}/2)$ to maximize the minimum area among all $\binom{n}{3}$ triangles formed by point triples. **Target:** $\Delta_{\min} = 0.03653$

System Prompt.

You are an expert computational geometer and optimization specialist with deep expertise in the Heilbronn triangle problem - a classical problem in discrete geometry that asks for the optimal placement of n points to maximize the minimum triangle area formed by any three points.

PROBLEM SPECIFICATION: Your task is to design and implement a constructor function that generates an optimal arrangement of exactly 11 points within or on the boundary of an equilateral triangle with vertices at $(0,0)$, $(1,0)$, and $(0.5, \sqrt{3}/2)$.

PERFORMANCE METRICS: 1. $\text{min_area_normalized}$: Area of the smallest triangle among all point triplets (PRIMARY OBJECTIVE - maximize) 2. combined_score : $\text{min_area_normalized} / 0.036529889880030156$ (BENCHMARK COMPARISON - maximize above 1.0) 3. eval_time : Function execution time in seconds (EFFICIENCY - minimize, but secondary to quality)

TECHNICAL REQUIREMENTS: - Determinism: Use fixed random seeds if employing stochastic methods for reproducibility - Error handling: Graceful handling of optimization failures or infeasible configurations

MINMAX DISTANCE

Task. Place $n = 16$ points in 2D Euclidean space (normalized to the unit square $[0, 1] \times [0, 1]$) to maximize the ratio $d_{\min}/d_{\max}$, where $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ for all $i \neq j$, $d_{\min} = \min_{i \neq j} d_{ij}$, and $d_{\max} = \max_{i \neq j} d_{ij}$. **Target:** $d_{\min}/d_{\max} = 1/\sqrt{12.889266112} \approx 0.2786$

System Prompt.

SETTING: You are an expert computational geometer and optimization specialist focusing on point dispersion problems. Your task is to evolve a constructor function that generates an optimal arrangement of exactly 16 points in 2D space, maximizing the ratio of minimum distance to maximum distance between all point pairs.

PROBLEM CONTEXT: - Target: Beat the AlphaEvolve benchmark of min/max ratio = $1/\sqrt{12.889266112} \approx 0.2786$ - Constraint: Points must be placed in 2D Euclidean space (typically normalized to unit square $[0, 1] \times [0, 1]$) - Mathematical formulation: For points $P_i = (x_i, y_i)$, $i = 1, \dots, 16$:

- Distance matrix: $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ for all $i \neq j$
- Minimum distance: $d_{\min} = \min\{d_{ij} : i \neq j\}$
- Maximum distance: $d_{\max} = \max\{d_{ij} : i \neq j\}$
- Objective: maximize $d_{\min}/d_{\max}$ subject to spatial constraints

PERFORMANCE METRICS: 1. min_max_ratio: $d_{\min}/d_{\max}$ ratio (PRIMARY OBJECTIVE – maximize) 2. combined_score: $\text{min_max_ratio} / 0.2786$ (progress toward beating AlphaEvolve benchmark) 3. eval_time: Execution time in seconds (balance accuracy vs. efficiency)

TECHNICAL REQUIREMENTS: - Reproducibility: Fixed random seeds for all stochastic components

PRISM — GPU MODEL PLACEMENT

Task. Improve the `compute_model_placement` function that assigns models to available GPUs. Given each model’s request rate, SLO, and memory footprint, and each GPU’s memory capacity `GPU_MEM_SIZE`, the per-GPU KV cache pressure is

$$\text{KVPR} = \frac{\sum_{m \in \text{GPU}} r_m / \text{SLO}_m}{\text{GPU_MEM_SIZE} - \sum_{m \in \text{GPU}} \text{size}_m}.$$

Minimize $\max_{\text{GPU}} \text{KVPR}$ subject to the memory feasibility constraint $\sum_{m \in \text{GPU}} \text{size}_m < \text{GPU_MEM_SIZE}$. **Target:** Minimize the maximum KVPR across all GPUs.

System Prompt.

You are an expert for model placement on GPUs. Your task is to improve a model placement algorithm by improving the function named `compute_model_placement` in the initial program that places models to available GPUs.

The algorithm must MINIMIZE the maximum KVPR across all GPUs while ensuring models can fit into the GPUs’ memory. Note that KVPR is the KV cache pressure for a GPU; it indicates how crowded a GPU is. For a specific GPU, its KVPR is computed as $\text{sum}(\text{model.req_rate}/\text{model.slo for model in models}) / (\text{GPU_MEM_SIZE} - \text{sum}(\text{model.model_size for model in models}))$, where `models` are the models assigned to this GPU.

The generated program should be as simple as possible and the code should execute correctly without errors.

EPLB — EXPERT PARALLELISM LOAD BALANCER FOR VLLM

Task. Improve the Mixture-of-Experts Expert Parallelism Load Balancer rearrangement algorithm (`rebalance_experts`) for vLLM. Given per-expert load metrics collected from

the vLLM server, produce an expert-to-device assignment (optionally with replicas of hot experts) that balances GPU load while keeping the algorithm itself fast, since perfect load balancing is NP-hard. **Target:** Improve load balance and reduce algorithm runtime over the baseline `rebalance_experts`.

System Prompt.

You are an expert programmer specializing in optimization algorithms. Your task is to improve the Mixture-of-Expert models Expert Parallelism Load Balancer (MoE EPLB) expert rearrangement algorithm.

This algorithm will take the load metrics recorded by the vLLM server, and rearrange the experts to balance the load. It can make replicas of some experts to achieve better load balancing.

Your goal will be two-fold:

1. Improve the algorithm to achieve better load balancing; while
2. Improve the algorithm to be more efficient, *i.e.* reduce the execution time of the algorithm itself, since perfect load balancing is NP-hard.

The current algorithm is implemented in the `rebalance_experts` function.

TXN SCHEDULING — DATABASE WORKLOADS

Task. Improve `get_best_schedule` to order database transactions so as to minimize total makespan. Each transaction is a sequence of read (r-k) and write (w-k) operations on keyed data items; read-write and write-write conflicts on the same key induce dependencies that delay execution. Only code inside `EVOLVE-BLOCK-START` / `EVOLVE-BLOCK-END` may change. **Target:** Minimize makespan on the provided JSON workloads.

System Prompt.

You are an expert in database transaction optimization. Only change code within `EVOLVE-BLOCK-START` and `EVOLVE-BLOCK-END`. Your task is to improve a scheduling function to find better schedules for transactional workloads made up of read and write operations to data items. There are conflicts between these transactions on items and reducing the delay of these conflicts will lead to schedules with lower makespan. Focus on improving the `get_best_schedule` function to find a schedule with as low makespan as possible.

TASK: Improve the `get_best_schedule` function to find optimal transaction schedules that minimize makespan for database workloads with read/write conflicts.

PROBLEM SPECIFICS: - **Input:** JSON workload with transactions such as

"txn0": "w-17 r-5 w-3 r-4 r-54 r-14 w-6 r-11 w-22 r-7 w-1 w-8 w-9 w-27 r-2 r-25" - **Operations:** Each transaction is a sequence of read (r-{key}) and write (w-{key}) operations on data items. - **Conflicts:** Read-write and write-write conflicts on the same key create dependencies between transactions. - **Goal:** Find a transaction ordering that minimizes total makespan.

SEARCH SUGGESTIONS: - **Greedy:** Try a greedy algorithm that iteratively picks the transaction that increases makespan the least. - Avoid relying only on heuristics like transaction length or number of writes; these do not correspond to the actual makespan of the schedule. Focus on evolving `get_best_schedule` to produce the best schedule possible with the lowest makespan. Explain step-by-step the reasoning process for your solution and how it will lead to a better schedule.

LLM SQL — PROMPT CACHING COLUMN REORDERING

Task. Given a pandas DataFrame `df` of text data, evolve the `Evolved` class so that, when an LLM processes rows sequentially, as much of the prefix as possible is reused from the previous row. For a column ordering C , $\text{PHC}(C) = \sum_r \text{hit}(C, r)$, where $\text{hit}(C, r) = \sum_{f \in \text{prefix}} |df[r][C[f]]|^2$ over matching prefix fields (zero if the first field mismatches). The algorithm must also be fast in wall-clock time. **Target:** Maximize

$\text{combined_score} = 0.95 \cdot \overline{\text{hit}} + 0.05 \cdot (12 - \min(12, \bar{t}))/12.$

System Prompt.

You are an expert in data optimization and LLM prompt caching. Your task is to evolve the existing Evolved class to maximize prefix hit count (PHC) for efficient LLM prompt caching.

Problem Context: - You are given a pandas DataFrame df with text data in rows and columns
- The goal is to reorder columns to maximize prefix reuse when processing rows sequentially -
Prefix reuse occurs when consecutive rows have matching values in the same column positions
- This reduces LLM computation costs by reusing cached prefixes

Objective: - Dual objective: (1) maximize prefix reuse across consecutive rows and (2) minimize end-to-end runtime of the algorithm. - Prefix reuse is defined as consecutive field values (starting from the first column) that are **exact matches** with the corresponding fields of the previous row. - The **hit score** of a row is the **sum of squares of the string lengths** of the matching prefix fields. - Combined score used for selection: $\text{combined_score} = 0.95 * \text{average_hit_rate} + 0.05 * (12 - \min(12, \text{average_runtime})) / 12$.

Required API (DO NOT CHANGE):

```
class Evolved(Algorithm):
    def reorder(
        self,
        df: pd.DataFrame,
        early_stop: int = 0,
        row_stop: int = None,
        col_stop: int = None,
        col_merge: List[List[str]] = [],
        one_way_dep: List[Tuple[str, str]] = [],
        distinct_value_threshold: float = 0.8,
        parallel: bool = True,
    ) -> Tuple[pd.DataFrame, List[List[str]]]:
```

Constraints: - Do not add/remove rows or columns - Different rows must be allowed to use different column orderings - Return a DataFrame with the same shape as input; use exact string matching - Preserve all existing method signatures and class structure
Simply return the optimized Evolved class, do not provide explanations.

IFBENCH AGENT SCAFFOLD — ONE-CALL SETTING

Task. Improve the Agent class to solve instruction-following tasks using a local LLM (Qwen3-8B). Each problem contains explicit constraints such as word count, format, forbidden words, language, and other task-specific requirements. The agent must satisfy all constraints in a single response under an exactly one-call budget. **Target:** Maximize loose constraint-satisfaction accuracy.

System Prompt.

You are an expert AI researcher improving an agent scaffold (the Agent class) that solves **instruction-following** tasks using a local LLM (Qwen3-8B).

The agent has a budget of EXACTLY 1 LLM call per problem. There is only one shot per problem. The evaluation metric is loose accuracy (fraction of constraints satisfied, 0.0–1.0). Each problem comes with a set of explicit constraints (*e.g.* word count, format, forbidden words, language, etc.). The agent must satisfy ALL constraints in a single response.

Some possible directions to explore for prompt engineering, you could also try other techniques (all within 1 LLM call):

1. System prompt clarity: precise instructions to follow ALL constraints exactly, no exceptions
2. Constraint-awareness framing: instruct the model to enumerate and verify each constraint
3. Few-shot examples: embed 1–2 worked examples showing constraint-compliant responses
4. Chain-of-thought in one pass: instruct model to reason about constraints then respond
5. Output anchoring: strong reminders about format, length, language, and forbidden content

Rules:

- `Agent.__init__` receives `query_llm` (callable). Do NOT change its signature.
- `forward(prompt: str)` must return `(response_str: str, cost: float)`.
- Do NOT add new imports outside the EVOLVE block.
- Do NOT call `query_llm` more than once — the evaluator strictly enforces the 1-call budget.

Maximize constraint satisfaction on the instruction-following dataset.

IFBENCH AGENT SCAFFOLD — THREE-CALL SETTING

Task. Improve the `Agent` class to solve instruction-following tasks using a local LLM (Qwen3-8B). Each problem contains explicit constraints such as word count, format, forbidden words, language, palindrome requirements, and other task-specific requirements. The agent may use up to three LLM calls per problem. **Target:** Maximize loose constraint-satisfaction accuracy.

System Prompt.

You are an expert AI researcher improving an agent scaffold (the `Agent` class) that solves **instruction-following** tasks using a local LLM (Qwen3-8B).

The agent has a budget of UP TO 3 LLM calls per problem. The evaluation metric is loose accuracy (fraction of constraints satisfied, 0.0–1.0). Each problem comes with a set of explicit constraints (e.g. word count, format, forbidden words, language, palindrome, etc.). The agent must satisfy ALL constraints.

Some possible directions to explore, you could also try other techniques (all within 3 LLM call):

1. System prompt clarity: explicit step-by-step instructions, chain of thought prompting, output format reminder
2. Constraint decomposition: enumerate and verify each constraint individually, then synthesize
3. Iterative refinement: ask the LLM to check its own answer and correct if wrong
4. Few-shot examples: include 1–2 worked examples in the prompt to anchor output format
5. Output anchoring: strong reminders about format, length, language, and forbidden content

Rules:

- `Agent.__init__` receives `query_llm` (callable). Do NOT change its signature.
- `forward(prompt: str)` must return `(response_str: str, cost: float)`.
- Do NOT add new imports outside the EVOLVE block.
- Do NOT call `query_llm` more than 3 times — the evaluator strictly enforces this budget.

Maximize constraint satisfaction on the instruction-following dataset.

XSTEST AGENT SCAFFOLD — ONE-CALL SETTING

Task. Improve the `Agent` class to classify user prompts as `safe` or `unsafe` using a local LLM (Qwen3-8B). The agent has exactly one LLM call per prompt, and voting is not allowed; the only lever is prompt engineering. **Target:** Maximize F1 score with `unsafe` as the positive class.

System Prompt.

You are an expert NLP researcher specializing in AI safety and content moderation. You are improving an agent scaffold (the `Agent` class) that classifies user prompts as “safe” or “unsafe” using a local LLM (Qwen3-8B).

The agent has a budget of EXACTLY 1 LLM call per prompt. Voting is NOT allowed. The only lever is prompt engineering: craft a single system+user prompt that handles all cases in one shot.

The evaluation metric is F1 score with “unsafe” as the positive class (maximize `combined_score = F1`).

Directions to explore (all within 1 LLM call):

1. System prompt clarity: precise instructions for edge cases (homonyms, fiction, metaphors)
 2. Few-shot examples: embed 2–4 labeled examples directly in the prompt
 3. Chain-of-thought in one pass: ask model to reason and verify then output label in a single response
 4. Structured output format: guide the model to a clear final label
 5. Domain-specific rules: explicit rules for technical language, roleplay, historical content
- The `extract_answer` function must reliably parse the final label. Do NOT modify the function signature of `forward`. Do NOT add more than 1 LLM call — the evaluator enforces this strictly and returns `score=0` for the entire example if the budget is exceeded. Maximize the `combined_score` (F1). You have multiple generations to explore.

XSTEST AGENT SCAFFOLD — THREE-CALL SETTING

Task. Improve the `Agent` class to classify user prompts as `safe` or `unsafe` using a local LLM (Qwen3-8B). The agent may use at most three LLM calls per prompt, allowing multi-step prompting, verification, or voting within the call budget. **Target:** Maximize F1 score with `unsafe` as the positive class.

System Prompt.

You are an expert NLP researcher specializing in AI safety and content moderation. You are improving an agent scaffold (the `Agent` class) that classifies user prompts as “safe” or “unsafe” using a local LLM (Qwen3-8B).

The agent has a budget of at most 3 LLM calls per prompt. The evaluation metric is F1 score with “unsafe” as the positive class (maximize `combined_score` = F1).

Directions to explore:

1. Chain-of-thought reasoning: ask the model to reason before classifying
2. Context-aware prompting: instruct the model to consider literal vs figurative meaning
3. Multi-step verification: first classify, then verify with a focused second call
4. Calibrated confidence: use voting or re-ranking across multiple temperatures
5. Targeted few-shot examples: include edge cases like homonyms or fiction framing
6. Decomposed reasoning: separate steps for intent detection and harm assessment

The `extract_answer` function must reliably parse the final label from the LLM response. Keep it robust. Do NOT modify the function signature of `forward`.

Maximize the `combined_score` (F1). You have multiple generations to explore.

H Prompt and Response Example for SEAEVO

Example prompt context for SEAEVO_{SHINKA} on TXN

System Prompt.

<task_system_prompt> # See Appendix G

Explain step-by-step the reasoning process for your solution and how this will lead to a better schedule.

Create a novel algorithm that draws inspiration from the provided context programs but implements a fundamentally different approach. Study the patterns and techniques from the examples, then design something new.

Before writing the program, reason through your approach step by step: # SA prompting

Step 1 — Diagnosis:

<DIAGNOSIS> Identify the key insights and techniques from the context programs worth borrowing. Then explain what makes the current solution underperform, and how combining or adapting those insights could address those weaknesses. </DIAGNOSIS>

Step 2 — Strategy:

<STRATEGY> The inspired but novel algorithmic strategy in 1-3 concise sentences. Explain what you are borrowing from the context and how you are applying it in a new way. </STRATEGY>
Step 3 — Implementation:

Provide the final implementation using the <NAME>, <DESCRIPTION>, and <CODE> format specified in the task.

User Message.

Current program

Here is the current program we are trying to improve (you will need to propose a new program with the same inputs and outputs as the original program, but with improved internal implementation):

<original_program> # Full program, omitted here for brevity

Here are the performance metrics of the program:

combined_score: 3968.2540

Reference Context # SER retrieved strategy context

Reference programs:

- (score=4016.0643): We will implement an Iterated Local Search (ILS) that combines a high-quality greedy insertion construction with a targeted local search focusing on the 'tail' transactions that define the makespan. The strategy uses greedy insertion to find a strong starting point and then iteratively applies a 'shaking' mechanism (multiple random swaps) followed by intensive hill-climbing to explore better neighborhoods.

- (score=3968.2540): The strategy is to combine the robust **Greedy Insertion** initialization from the current program with a more aggressive **Iterated Local Search (ILS)** that prioritizes **Insertion Moves** over swaps. I will implement a multi-restart approach where each restart begins with a greedy-randomized insertion sequence, followed by an intensification phase that uses targeted insertion heuristics (focusing on the tail of the schedule) and a shaking phase to escape local minima.

Guidance # SLN generated landscape guidance

1. **STOP using "Blind" Local Search:** Cease purely random swaps. Implement a "Critical Path" heuristic where the algorithm identifies which transaction's finish time equals the total makespan and prioritizes moving that specific transaction earlier.
2. **Shift to "Greedy Insertion" Construction:** Instead of building sequences from left-to-right, try starting with a partial sequence and "inserting" new transactions into the position that minimizes the resulting makespan ($O(N^2)$ complexity but much higher quality).
3. **Implement a "Shaking" Phase:** To break the 3350 plateau, adopt a Variable Neighborhood Search (VNS) or Iterated Local Search (ILS) that performs 3-5 random swaps once a local optimum is reached, then restarts the hill climber.
4. **Prioritize Refinement over Construction:** The history shows that $O(N^2)$ greedy construction is expensive. Use a faster heuristic for the first 10% of the budget, then spend 90% on high-quality insertion-based local search.

Task

Analyze the weaknesses of the current solution and rewrite it to maximize the score.

You MUST respond using a short summary name, description and the full code:

<NAME> A shortened name summarizing the code you are proposing. Lowercase, no spaces, underscores allowed. </NAME>

<DESCRIPTION> [SUMMARY: First provide exactly 2 concise sentences summarizing the core algorithmic approach you are proposing.] [RATIONALE: Then provide detailed argumentation for why this approach will improve performance.] </DESCRIPTION>

<CODE> “python # Your complete rewritten program here. ““ </CODE>

* Keep the markers "EVOLVE-BLOCK-START" and "EVOLVE-BLOCK-END" in the code. Do not change the code outside of these markers.

* Make sure your rewritten program maintains the same inputs and outputs as the original program, but with improved internal implementation.

* Make sure the file still runs after your changes.

* Ensure all necessary imports are included at the top of the code (e.g., numpy, math, scipy). Missing imports will cause failure.

* Use the <DIAGNOSIS>, <STRATEGY>, <NAME>, <DESCRIPTION>, and <CODE> delimiters to structure your response. It will be parsed afterwards.

Example response for SEAEVO_{SHINKA} on TXN

<DIAGNOSIS>

The current solution uses a high-cost greedy insertion for initialization and a somewhat unfocused Iterated Local Search. Its main weakness is the lack of "critical path" awareness; it treats all transactions equally during the local search phase rather than focusing on the bottlenecks that actually determine the makespan. Furthermore, the search budget is spent inefficiently on exhaustive insertion checks for every new transaction during initialization, leaving less time for refining the global structure.

</DIAGNOSIS>

<STRATEGY>

I will implement a "Critical-Focus Hill Climber" that identifies the "bottleneck" transaction—the one whose finish time defines the current makespan—and prioritizes moving it earlier in the sequence. To maintain diversity and escape local optima, I'll combine this with a fast randomized greedy construction and a periodic "shaking" mechanism that relocates multiple transactions simultaneously.

</STRATEGY>

<NAME> critical_path_bottleneck_ils </NAME>

<DESCRIPTION>

[SUMMARY: This approach uses a fast randomized-greedy construction followed by an Iterated Local Search that specifically targets the bottleneck transaction responsible for the current makespan. It identifies which transaction finishes last and tries to shift it to various preceding positions to break dependencies.] [RATIONALE: By focusing on the transaction that determines the makespan, we prune the search space significantly compared to random swaps or insertions. The addition of a shaking phase ensures that when the bottleneck cannot be improved locally, the algorithm jumps to a new neighborhood, while the use of a faster construction phase preserves more of the execution budget for high-impact local refinements.]

</DESCRIPTION>

<CODE>

<revised_code> # Full generated program, omitted here for brevity

</CODE>

Figure 10: Illustrative prompt and generated response on the TXN task. The prompt combines the task instruction, current program, SA prompting, SER-retrieved strategy context, and SLN-generated landscape guidance; the generated response is parsed into diagnosis, strategy description, and revised program. Explanatory comments are shown in gray.

SLN Prompt Context

System Prompt.

You are an expert research scientist analysing the search landscape of an automated algorithm evolution experiment. Your role is to identify patterns, bottlenecks, and unexplored directions to guide the next phase of search. Output exactly these four XML tags in order: <effective>...</effective> <saturated>...</saturated> <unexplored>...</unexplored> <guidance>...</guidance>

User Message.

Evolution History Below is a record of every generation so far (generation number, score, algorithmic idea, and diagnosis of the previous program's failures).

Generation 1 (score=0.1258) **Idea:** Vectorize the entire rebalancing pipeline by replacing Python loops with batch-aware PyTorch operations and a prioritized heap-free greedy assignment. We use a pre-allocated tensor-based greedy approach that determines all replicas for all layers simultaneously, significantly reducing execution time while maintaining high packing quality. **Diagnosis:** The current implementation suffers from significant performance bottlenecks and suboptimal load distribution. Specifically: 1. **Inefficient CPU Loops:** The 'balanced_packing' and 'replicate_experts' functions rely on nested Python loops that iterate over layers and experts, leading to high execution overhead. 2. **Greedy Suboptimality:** The Longest Processing Time (LPT) greedy approach in 'replicate_experts' is performed sequentially, which is slow. 3. **Redundant Memory Operations:** Frequent tensor reshaping and gathering on the CPU create unnecessary overhead. 4. **Limited Parallelism:** The algorithm processes

layers independently in a loop instead of leveraging PyTorch's vectorized operations across the batch (layer) dimension.

```
### Generation 2 (score=0.1129) **Idea:** Replace the iterative greedy replication with a fully vectorized approach using 'torch.topk' to identify high-load experts across all layers simultaneously. Implement a more robust greedy packing algorithm that uses cumulative sum and searchsorted to distribute loads across packs in a single pass, improving both speed and balancing quality. **Diagnosis:** The current implementation suffers from two primary bottlenecks: a sequential greedy replication loop in 'replicate_experts_vec' and a suboptimal packing strategy in 'balanced_packing_vec'. The replication loop iterates 'num_redundant' times for every layer, which is extremely slow when scaled to many layers or high redundancy. Furthermore, the "snake-like" packing heuristic, while fast, does not account for the high variance in expert loads, leading to poor load balancing. The hierarchical logic also introduces unnecessary complexity and potential overhead in the gathering/scattering of indices across nodes and GPUs. ... # We show an abridged prompt context for readability
```

SLN Generated Guidance at Generation 10

<effective>

- Vectorization of the replication logic: moving away from $O(N)$ loops to batch-calculated load-proportional allocation has consistently maintained baseline performance while reducing overhead.
- Snake-sort / Zig-zag interleaving: a robust, low-complexity alternative to iterative greedy packing, providing stable results across multiple generations.
- Priority-based allocation: using sorted weights (LPT-inspired) to assign experts to packs consistently outperforms random or sequential assignment.

</effective>

<saturated>

- Iterative greedy loops: repeated attempts to fix $O(N)$ loops via minor logical tweaks have plateaued; the overhead remains the primary bottleneck.
- Hierarchical constraints: rigid Node $\rightarrow$ Group $\rightarrow$ GPU boundaries limit the global optimisation space.
- Simple Zig-Zag packing: the static snake-sort pattern has reached its limit on highly skewed expert-weight distributions.

</saturated>

<unexplored>

- Cumulative Load-Aware Vectorization: prefix-sum-based bin packing without loops.
- Differentiable Sorting / Ranking: soft-sorting relaxation for load balancing.
- Dynamic Partitioning: variable experts-per-GPU within memory limits.
- Memory-Centric Optimization: eliminate `.clone()` and `.cpu()` calls via a single fused Torch operation.

</unexplored>

<guidance>

1. Abandon "Snake-Sort"; try a Water-Filling algorithm via vectorised prefix sums and binary search.
2. Remove all `.cpu()` and `.item()` calls to prevent PCIe latency.
3. Flatten the hierarchical structure: assign N experts to M GPUs globally, then reshape.
4. Use `torch.topk / sort` on mean-load residuals for a single vectorised rounding step.

</guidance>

SLN Generated Guidance at Generation 50

<effective>

- **D'Hondt (Jefferson) Method:** divisor-based approach consistently outperformed proportional-fair and water-filling heuristics.
- **Prefix-Sum Load Partitioning:** `torch.cumsum + torch.searchsorted` for GPU boundaries outperforms Snake-Sort.
- **Full Vectorization:** eliminating all Python-level loops is essential to avoid CPU-GPU synchronisation overhead.
- **Boundary Expert Optimisation:** midpoint / cost-evaluation logic for experts straddling the load boundary is critical for minimising max-load.

</effective>

<saturated>

- Snake-Sort / Zig-Zag: plateaued around 0.12–0.14; cannot handle skewed distributions.
- Hierarchical Node-to-GPU logic: fragments the optimisation space.
- Greedy Argmin/Argmax loops: replaced by vectorised approximations.

</saturated>

<unexplored>

- Intra-Layer Load Swapping (Local Search): single-pass vectorised swap check after initial prefix-sum assignment.
- Multi-Objective Optimisation: penalty for cross-node expert moves in the D'Hondt scoring.
- Look-Ahead Residual Allocation: unify replication factor and bin-gap fitting.
- Weighted Round-Robin with Capacity Constraints: vectorised apportionment with hard per-GPU expert limits.

</unexplored>

<guidance>

1. Implement vectorised comparison for the straddle expert: $\min(\max(L_i + w, L_{i+1}), \max(L_i, L_{i+1} + w))$.
2. Use `torch.arange + searchsorted` on replica offsets for $O(1)$ rank generation.
3. Avoid iterative binary search; D'Hondt with optimised `topk` is faster.
4. Keep the weight tensor on-device; a single `.cpu()` call negates algorithmic gains.

</guidance>

SLN Generated Guidance at Generation 100

<effective>

- **D'Hondt Method:** proportional allocation outperformed simple rounding or greedy loops.
- **Prefix-Sum Load Partitioning:** continuous load boundaries before discretisation significantly improved balance.
- **Vectorised Rank Generation:** `torch.cumsum` on logical expert transitions replaced $O(N^2)$ masks.
- **Boundary-Cost Optimisation:** explicit minimax check at each GPU boundary outperformed midpoint heuristics.

</effective>

<saturated>

- Snake / Zig-Zag heuristics: plateaued at ~ 0.12 – 0.14 .
- Midpoint-based partitioning: consistently fails to reach the minimax optimum.
- Independent layer optimisation: local imbalances accumulate into a global bottleneck.
- Iterative greedy replication: replaced by vectorised $\text{top-}k$ / divisor methods.

</saturated>

<unexplored>

- Global Multi-Layer Load Balancing: allow load carry-over across layers.
- Karmarkar–Karp Vectorisation: differencing algorithm for multi-way partition.
- Communication-Aware Hierarchical Constraints: cross-node transfer cost in the objective.
- Hardware-Aware Kernel Fusion: single CUDA extension or `torch.compile` to eliminate intermediate tensor allocations.

</unexplored>

<guidance>

1. Implement a Global Load Tracker: accumulate residual error from layer i and offset target boundaries for layer $i+1$.
2. Try Karmarkar–Karp differencing (Largest Differencing Method) for partitioning.
3. Eliminate `sort/argsort` in metadata; use `scatter_add + cumsum` on pre-sorted indices for $O(N)$ rank/mapping.
4. Replace midpoint heuristics with explicit Minimax boundary checks.
5. Remove all PCIe syncs; rewrite any `weight.cpu()` logic using `torch.where` or `torch.bucketize`.

</guidance>

Figure 11: Illustrative SLN prompt context and generated landscape guidance on the EPLB task. The prompt summarizes the evolution history up to a given generation, and SLN returns structured guidance over effective, saturated, and underexplored strategy families.

I Discovered Program

```

1  import random
2  import time
3  import math
4
5  GPU_MEM_SIZE = 80 # GB
6
7  # EVOLVE-BLOCK-START
8  def compute_model_placement(gpu_num, models):
9      if not models:
10         return {i: [] for i in range(gpu_num)}
11
12     start_time = time.time()
13
14     def get_gpu_metrics(gpu_models):
15         used_mem = sum(m.model_size for m in gpu_models)
16         pressure = sum(m.req_rate / m.slo for m in gpu_models)
17         rem_mem = GPU_MEM_SIZE - used_mem
18         kvpr = pressure / rem_mem if rem_mem > 1e-7 else 1e15
19         return kvpr, rem_mem, pressure
20
21     # 1. Diverse Greedy Initialization
22     best_placement = {i: [] for i in range(gpu_num)}
23     min_max_kvpr = float('inf')
24
25     # Try 3 different sorting strategies for the initial greedy pass
26     strategies = [
27         lambda m: (m.req_rate / m.slo) / max(0.01, m.model_size),
28         lambda m: (m.req_rate / m.slo),
29         lambda m: m.model_size
30     ]
31
32     for strat in strategies:
33         if time.time() - start_time > 0.2: break
34         curr_p = {i: [] for i in range(gpu_num)}
35         curr_mems = [0.0] * gpu_num
36         sorted_models = sorted(models, key=strat, reverse=True)
37

```



```

38 fail = False
39 for m in sorted_models:
40     best_g, best_v = -1, float('inf')
41     for g_id in range(gpu_num):
42         if curr_mems[g_id] + m.model_size <= GPU_MEM_SIZE - 1e-5:
43             curr_p[g_id].append(m)
44             kv, _, _ = get_gpu_metrics(curr_p[g_id])
45             if kv < best_v:
46                 best_v, best_g = kv, g_id
47             curr_p[g_id].pop()
48
49     if best_g == -1:
50         # Fallback: place in GPU with most room
51         best_g = max(range(gpu_num), key=lambda g: GPU_MEM_SIZE -
52             curr_mems[g])
53         if curr_mems[best_g] + m.model_size > GPU_MEM_SIZE: fail = True
54
55     curr_p[best_g].append(m)
56     curr_mems[best_g] += m.model_size
57
58     if not fail:
59         max_kv = max(get_gpu_metrics(curr_p[g])[0] for g in range(gpu_num))
60         if max_kv < min_max_kvpr:
61             min_max_kvpr = max_kv
62             best_placement = {g: list(curr_p[g]) for g in range(gpu_num)}
63
64 # 2. Targeted LNS Refinement
65 curr_p = {g: list(best_placement[g]) for g in range(gpu_num)}
66
67 while time.time() - start_time < 0.92:
68     # Identify bottleneck GPUs
69     stats = []
70     for g in range(gpu_num):
71         kv, rem, pres = get_gpu_metrics(curr_p[g])
72         stats.append({'id': g, 'kvpr': kv, 'rem': rem, 'pres': pres})
73
74     stats.sort(key=lambda x: x['kvpr'], reverse=True)
75     max_kv = stats[0]['kvpr']
76
77     # Destroy: Remove models from top pressured GPUs
78     removed_models = []
79     # Target top 1-2 GPUs
80     num_to_disrupt = min(2, gpu_num)
81     for i in range(num_to_disrupt):
82         target_g = stats[i]['id']
83         if not curr_p[target_g]: continue
84
85         # Remove 1-3 models from each target GPU
86         num_m = random.randint(1, min(3, len(curr_p[target_g])))
87         # Prefer removing models with high pressure
88         curr_p[target_g].sort(key=lambda m: m.req_rate/m.slo, reverse=True)
89         for _ in range(num_m):
90             if curr_p[target_g]:
91                 removed_models.append(curr_p[target_g].pop(0))
92
93     # Repair: Re-insert removed models using randomized regret/best-fit
94     random.shuffle(removed_models)
95     temp_p = {g: list(curr_p[g]) for g in range(gpu_num)}
96     possible = True
97
98     for m in removed_models:
99         candidates = []
100         for g_id in range(gpu_num):
101             used = sum(mod.model_size for mod in temp_p[g_id])
102             if used + m.model_size <= GPU_MEM_SIZE - 1e-5:
103                 temp_p[g_id].append(m)
104                 kv, _, _ = get_gpu_metrics(temp_p[g_id])
105                 candidates.append((kv, g_id))
106                 temp_p[g_id].pop()
107
108         if not candidates:
109             possible = False
110             break
111
112     # Pick from top 2 candidates to allow exploration
113     candidates.sort()
114     idx = 0 if (len(candidates) == 1 or random.random() > 0.2) else 1
115     best_g = candidates[idx][1]

```



```

115         temp_p[best_g].append(m)
116
117     if possible:
118         new_max_kv = max(get_gpu_metrics(temp_p[g])[0] for g in range(
119             gpu_num))
120         # Acceptance: Better or slightly worse (to escape local optima)
121         if new_max_kv < min_max_kvpr:
122             min_max_kvpr = new_max_kv
123             best_placement = {g: list(temp_p[g]) for g in range(gpu_num)}
124             curr_p = {g: list(temp_p[g]) for g in range(gpu_num)}
125         elif new_max_kv < max_kv * 1.05: # Soft acceptance
126             curr_p = {g: list(temp_p[g]) for g in range(gpu_num)}
127         else:
128             # If repair failed, revert curr_p to best known to try a different
129             # destruction
130             curr_p = {g: list(best_placement[g]) for g in range(gpu_num)}
131
132     return best_placement
133
134 # EVOLVE-BLOCK-END
135
136 if __name__ == "__main__":
137     # Test the algorithm
138
139     from evaluator import generate_test_gpu_models
140     from evaluator import calculate_kvcache_pressure
141     from evaluator import safe_float
142     import numpy as np
143
144     test_cases = generate_test_gpu_models()
145     all_kvpr = []
146     for i, (gpu_num, gpu_models) in enumerate(test_cases):
147         results = compute_model_placement(gpu_num, gpu_models)
148         max_kvpr = calculate_kvcache_pressure(results)
149         all_kvpr.append(safe_float(max_kvpr))
150
151     avg_kvpr = np.mean(all_kvpr)
152     if avg_kvpr != 0:
153         avg_kvpr = 1.0 / avg_kvpr
154
155     print(f"Max KVPR: {avg_kvpr:.3f}")

```

Figure 12: Best program discovered by SEAEVO_{SHINKA} with Gemini-3-Flash in Prism.

```

1  # EVOLVE-BLOCK-START
2
3  import torch
4
5  def rebalance_experts(
6      weight: torch.Tensor,
7      num_replicas: int,
8      num_groups: int,
9      num_nodes: int,
10     num_gpus: int,
11 ) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
12     """
13     Expert-parallelism load balancer using Greedy Utility Allocation and
14     Fast Index-Based Metadata Construction.
15     """
16     device = weight.device
17     num_layers, num_experts = weight.shape
18     weight = weight.float()
19
20     # 1. Marginal Gain Allocation
21     # Allocate replicas to experts with the highest marginal utility (weight /
22     # current_replicas)
23     if num_replicas > num_experts:
24         # Vectorized marginal utility calculation
25         # We need to pick (num_replicas - num_experts) additional replicas per
26         # layer
27         logcnt = torch.ones((num_layers, num_experts), device=device, dtype=
28             torch.int64)

```

```

26     num_extra = num_replicas - num_experts
27
28     # Iterative greedy is slow, use a closed-form approximation for top-k
29     # utilities
30     # utility(expert, k) = weight / k. We want top num_replicas across all k
31     k_range = torch.arange(1, num_replicas + 1, device=device).float()
32     all_utilities = weight.unsqueeze(-1) / k_range # [L, E, R]
33
34     # Flatten E and R to pick the best combinations per layer
35     flat_utils = all_utilities.view(num_layers, -1)
36     _, topk_indices = torch.topk(flat_utils, num_replicas, dim=1)
37
38     # Convert flat indices back to expert indices
39     expert_indices = torch.div(topk_indices, num_replicas, rounding_mode='
40     floor')
41
42     logcnt = torch.zeros((num_layers, num_experts), device=device, dtype=
43     torch.int64)
44     logcnt.scatter_add_(1, expert_indices, torch.ones_like(expert_indices))
45 else:
46     logcnt = torch.ones((num_layers, num_experts), device=device, dtype=
47     torch.int64)
48
49 # 2. Global Stream Partitioning
50 # Calculate weight of each physical replica
51 rep_weights = weight / logcnt.clamp(min=1).float()
52
53 # Flatten all possible physical replicas across all layers
54 # We use a fixed order per layer to ensure stable mapping
55 flat_phy2log = torch.repeat_interleave(
56     torch.arange(num_experts, device=device).expand(num_layers, num_experts)
57     ,
58     logcnt.flatten()
59 )
60
61 # Weights of each replica in the flattened stream
62 flat_phy_weights = rep_weights.flatten()[torch.repeat_interleave(
63     torch.arange(num_layers * num_experts, device=device),
64     logcnt.flatten()
65 )]
66
67 # Partition the global stream across GPUs
68 cum_weights = torch.cumsum(flat_phy_weights, dim=0)
69 total_weight = cum_weights[-1]
70 gpu_boundaries = torch.linspace(0, total_weight.item(), num_gpus + 1, device
71 =device)[1:-1]
72 # This assigns every physical replica to a GPU globally
73 global_gpu_ids = torch.searchsorted(gpu_boundaries, cum_weights)
74
75 # To return [num_layers, num_replicas], we need to reshape back.
76 # The current logic requires each layer to have exactly num_replicas.
77 # We maintain the layer-wise structure but the order within the layer is
78 # determined by the global GPU assignment to minimize cross-GPU imbalance.
79 final_phy2log = flat_phy2log.view(num_layers, num_replicas)
80 global_gpu_ids = global_gpu_ids.view(num_layers, num_replicas)
81
82 # Sort within each layer by GPU ID to group experts belonging to the same
83 GPU
84 sort_idx = torch.argsort(global_gpu_ids, dim=1, stable=True)
85 final_phy2log = final_phy2log.gather(1, sort_idx)
86
87 # 3. Fast Metadata Construction (O(N))
88 # Calculate phyrank without one-hot: rank within the layer for that logical
89 expert
90 # We use a trick: cumsum on a flattened version with expert offsets
91 max_reps = int(logcnt.max().item())
92 expert_offsets = torch.arange(num_layers * num_experts, device=device) * (
93     max_reps + 1)
94
95 # For each replica in final_phy2log, calculate its unique ID (layer, expert)
96 layer_offsets = torch.arange(num_layers, device=device).view(num_layers, 1)
97 * num_experts
98 flat_expert_ids = final_phy2log + layer_offsets
99
100 # To get ranks: we need to know if this is the 1st, 2nd... occurrence of
101 flat_expert_id in the layer
102 # Since experts are grouped by layer, we can use a local cumsum approach

```

```

92     # A robust way is using sort + cumsum, but since max_reps is small, we use a
93     # faster method
94     # We calculate phyrank by tracking occurrences
95     phyrank = torch.zeros_like(final_phy2log)
96     for r in range(num_replicas):
97         # This is slightly iterative but num_replicas is small (\textit{e.g.}
98         # 128)
99         # and it avoids the  $O(L \times E \times R)$  memory explosion
100        curr_experts = final_phy2log[:, r]
101        # Count how many times each expert has appeared before this column
102        if r > 0:
103            # Mask of experts seen in previous columns
104            prev_experts = final_phy2log[:, :r]
105            phyrank[:, r] = (prev_experts == curr_experts.unsqueeze(1)).sum(dim
106            =1)
107
108        # Construct log2phy: [L, E, MaxR]
109        log2phy = torch.full((num_layers, num_experts, max_reps), -1, dtype=torch.
110        int64, device=device)
111        l_idx = torch.arange(num_layers, device=device).view(num_layers, 1)
112
113        # Single-pass scatter to fill log2phy
114        # log2phy[l, e, rank] = physical_slot
115        scatter_idx = (l_idx * (num_experts * max_reps) +
116        final_phy2log * max_reps +
117        phyrank)
118
119        physical_slots = torch.arange(num_replicas, device=device).expand(num_layers
120        , num_replicas)
121        log2phy.view(-1).scatter_(0, scatter_idx.flatten(), physical_slots.flatten()
122        )
123
124    return final_phy2log, log2phy, logcnt
125
126# EVOLVE-BLOCK-END
127
128__all__ = ["rebalance_experts"]

```

Figure 13: Best program discovered by SEAEVO_{SHINKA} with Gemini-3-Flash in EPLB.

```

1  # EVOLVE-BLOCK-START
2  import numpy as np
3  from scipy.optimize import minimize
4
5  def circle_packing21() -> np.ndarray:
6      """
7      Places 21 non-overlapping circles inside a rectangle of perimeter 4 (w+h=2)
8      to maximize the sum of their radii.
9      """
10     n_circles = 21
11     best_sum = 0
12     best_circles = np.zeros((n_circles, 3))
13
14     # Seeds for reproducibility
15     seeds = [42, 123, 999, 7, 88]
16
17     def objective(vars):
18         # vars: [w, x0, y0, r0, x1, y1, r1, ...]
19         # We want to maximize sum(radii), so minimize -sum(radii)
20         radii = vars[3::3]
21         return -np.sum(radii)
22
23     def constraints(vars, n):
24         w = vars[0]
25         h = 2.0 - w
26         coords = vars[1:]
27         res = []
28
29         # Boundary constraints: x-r >= 0, x+r <= w, y-r >= 0, y+r <= h
30         for i in range(n):
31             x, y, r = coords[3*i], coords[3*i+1], coords[3*i+2]
32             res.append(x - r)
33             res.append(w - (x + r))
34             res.append(y - r)

```

```

35         res.append(h - (y + r))
36         res.append(r) # r > 0
37
38         # Non-overlap constraints: dist >= r1 + r2
39         for i in range(n):
40             xi, yi, ri = coords[3*i], coords[3*i+1], coords[3*i+2]
41             for j in range(i + 1, n):
42                 xj, yj, rj = coords[3*j], coords[3*j+1], coords[3*j+2]
43                 # (xi-xj)**2 + (yi-yj)**2 - (ri+rj)**2 >= 0
44                 res.append(np.sqrt((xi-xj)**2 + (yi-yj)**2 + 1e-9) - (ri + rj))
45         return np.array(res)
46
47     # Optimization Bounds
48     # w: [0.5, 1.5], x: [0, 1.5], y: [0, 1.5], r: [0, 0.5]
49     bounds = [(0.5, 1.5)] + [(0, 1.5), (0, 1.5), (0, 0.5)] * n_circles
50
51     for seed in seeds:
52         np.random.seed(seed)
53
54         # Initialization: Start with a rectangle near square and a grid-ish
55         # layout
56         w_init = 1.0
57         h_init = 1.0
58
59         # Create a jittered grid for 21 circles (approx 4x5 or 5x5)
60         # We'll place them and assign a small initial radius
61         x_init = np.linspace(0.1, 0.9, 5)
62         y_init = np.linspace(0.1, 0.9, 5)
63         grid_x, grid_y = np.meshgrid(x_init, y_init)
64         pts = np.vstack([grid_x.ravel(), grid_y.ravel()]).T
65         np.random.shuffle(pts)
66         pts = pts[:n_circles]
67
68         r_init = 0.05 * np.ones((n_circles, 1))
69         initial_vars = np.zeros(1 + 3 * n_circles)
70         initial_vars[0] = w_init
71         for i in range(n_circles):
72             initial_vars[1 + 3*i] = pts[i, 0]
73             initial_vars[1 + 3*i + 1] = pts[i, 1]
74             initial_vars[1 + 3*i + 2] = r_init[i]
75
76         # Optimization
77         res = minimize(
78             objective,
79             initial_vars,
80             method='SLSQP',
81             constraints={ 'type': 'ineq', 'fun': lambda v: constraints(v,
82                             n_circles)},
83             bounds=bounds,
84             options={'maxiter': 150, 'ftol': 1e-6}
85         )
86
87         if res.success or res.fun < 0:
88             current_sum = -res.fun
89             if current_sum > best_sum:
90                 best_sum = current_sum
91                 # Reshape result
92                 final_w = res.x[0]
93                 final_coords = res.x[1:].reshape((n_circles, 3))
94                 best_circles = final_coords
95
96         return best_circles
97
98     # EVOLVE-BLOCK-END

```

Figure 14: Best program discovered by SEAEVO_{SHINKA} with Gemini-3-Flash in Circle Packing (Rect).

```

1  import re
2  import json
3  from typing import Callable, Tuple
4  # EVOLVE-BLOCK-START
5  class Agent:
6      def __init__(self, query_llm: Callable, temperature: float = 0.0):

```

```

7         # Temperature 0.0 is critical for strict instruction following and
8         reproducibility
9         self.query_llm = query_llm
10        self.temperature = temperature
11
12    def forward(self, prompt: str) -> Tuple[str, float]:
13        system_prompt = (
14            "You are a precision instruction-following engine. You must satisfy"
15            "ALL constraints perfectly.\n"
16            "Use the following internal reasoning protocol before providing the"
17            "response:\n\n"
18            "1. <PLANNING_SCRATCHPAD>:\n"
19            "    - List all explicit constraints (length, format, forbidden words"
20            "    , etc.).\n"
21            "    - FORBIDDEN CONTENT: Identify any forbidden words/topics and"
22            "    assign them IDs (e.g., [F1], [F2]). NEVER type the actual"
23            "    forbidden words in this scratchpad.\n"
24            "    - STRUCTURE: Define the number of sentences/paragraphs. For each"
25            "    , state the intended first word and last word (Bookending).\n"
26            "    - COUNTING: If a word/character count is required, perform a"
27            "    preliminary count here.\n"
28            "2. <FINAL_RESPONSE>: Provide the output that satisfies every"
29            "constraint. Do not include any meta-talk or explanations.\n\n"
30            "EXAMPLE:\n"
31            "USER: Write a 4-word sentence about a cat without using the word 'meow'.\n"
32            "<PLANNING_SCRATCHPAD>:\n"
33            "    - Topic: Cat\n"
34            "    - Length: Exactly 4 words\n"
35            "    - Forbidden: [F1] (the 'm' word)\n"
36            "    - Plan: Sentence 1 starts with 'The', ends with 'sleeps'. \n"
37            "<FINAL_RESPONSE>:\n"
38            "The fluffy cat sleeps.\n"
39            "END_EXAMPLE"
40        )
41
42        user_input = (
43            f"REQUEST:\n{prompt}\n\n"
44            "Strictly follow the <PLANNING_SCRATCHPAD> and <FINAL_RESPONSE>"
45            "protocol."
46        )
47
48        response, cost = self.query_llm(
49            prompt=user_input,
50            system=system_prompt,
51            temperature=self.temperature
52        )
53
54        # Robust extraction focusing on the <FINAL_RESPONSE> tag
55        final_output = ""
56        if "<FINAL_RESPONSE>" in response:
57            # Take the content after the last occurrence of the tag to avoid
58            issues if the model repeats it
59            parts = response.split("<FINAL_RESPONSE>")
60            final_output = parts[-1].strip()
61            # Remove closing tag if present
62            if "</FINAL_RESPONSE>" in final_output:
63                final_output = final_output.split("</FINAL_RESPONSE>")[0].strip()
64        else:
65            # Fallback for models that might use lowercase or skip tags
66            if "<FINAL_RESPONSE" in response.upper():
67                tag_start = response.upper().find("<FINAL_RESPONSE")
68                content_start = response.find(">", tag_start) + 1
69                final_output = response[content_start:].strip()
70            else:
71                # If tags are totally missing, take the last block of text
72                parts = response.split('\n\n')
73                final_output = parts[-1].strip()
74
75        # Final cleanup of common LLM artifacts
76        # Remove markdown code blocks
77        final_output = re.sub(r'~`~\w*\n', '', final_output)
78        final_output = re.sub(r'~`~$', '', final_output).strip()
79
80        # Remove any leading colons or labels the model might have prepended
81        final_output = re.sub(r'^(Result|Output|Answer|Response):~s*', '',
82                               final_output, flags=re.IGNORECASE).strip()

```

```
71
72     # Remove any trailing "END_EXAMPLE" or similar hallucinated markers
73     final_output = re.sub(r'\n?END_EXAMPLE$', '', final_output).strip()
74
75     return final_output, cost
76 # EVOLVE-BLOCK-END
```

Figure 15: Best program discovered by SEAEVO_{SHINKA} with Gemini-3-Flash in IFBench.